UltraProofs: Scalable Reed–Solomon Code Commitment

Yanpei Guo
National University of Singapore
guo.yanpei@u.nus.edu

Alex Luoyuan Xiong
National University of Singapore
alex.xiong.tech@gmail.com

Wenjie Qu
National University of Singapore
wenjiequ@u.nus.edu

Jiaheng Zhang
National University of Singapore
jhzhang@nus.edu.sg

***Abstract*—Reed–Solomon (RS) codes underpin a wide range of cryptographic protocols, from verifiable secret sharing (VSS) to blockchain data availability (DA). An RS code commitment enables a prover to distribute codeword fragments among many parties while allowing each recipient to verify that its fragment is consistent with a RS code. Existing constructions either rely on homomorphic polynomial commitments, which incur redundant commitments and expensive group operations, or use FRI-based interactive proofs, which achieve fast proving but suffer from prohibitively large communication.**

We present UltraProofs, a new RS code commitment framework that achieves linear-time proof generation while remaining compatible with any multilinear polynomial commitment. Our key technical contribution is an evaluation-consolidation protocol that reduces n evaluation proofs at distinct RS points to a single randomized evaluation, eliminating redundant commitments and removing the need for homomorphic structure. We further design a tailored multilinear PCS, LightLigero, which achieves $O(\lambda \log n)$ proof size and maintains $O(n \log n)$ prover time.

UltraProofs attains asymptotically optimal prover complexity with concrete speedups in practice: in VSS, it reduces prover time by $2.4\times$ and proof size by $4\times$ compared to HydraProofs (S&P'25); in DA, it cuts per-node communication by up to $2\sim5\times$ relative to FRIDA (Crypto'24) while retaining similar prover cost. By decoupling verifiable RS encoding from any specific PCS instantiation, UltraProofs provides a flexible, efficient, and modular foundation for large-scale verifiable storage and distributed cryptographic systems.

1. Introduction

Reed–Solomon (RS) codes [1] are foundational erasure codes that provide data integrity and redundancy in distributed storage [2], [3], secret sharing [4], [5], [6], consensus [7], [8], [9], and blockchain data availability [10], [11], [12]. Formally, an RS code $\mathsf{RS}[k, n, \mathbb{F}] : \mathbb{F}^k \to \mathbb{F}^n$ $(k < n)$ encodes a message vector $\vec{m} \in \mathbb{F}^k$ into a *codeword*

$$\vec{c} = (f(\omega^0), f(\omega^1), \ldots, f(\omega^{n-1})),$$

where $f(X)$ is the unique degree-$(k-1)$ polynomial interpolating $\vec{m}$ and $\omega \in \mathbb{F}$ is an n-th root of unity. Each evaluation $f(\omega^i)$ is typically dispersed to a distinct user or storage node, enabling any subset of k evaluations to reconstruct $\vec{m}$.

In cryptographic applications [13], [14], [15], however, the disperser may be *malicious* and distribute inconsistent or invalid shares. To ensure verifiability, the notion of an *RS code commitment* [15], [16] extends classical RS dispersal with cryptographic guarantees of *binding* and *consistency*. Intuitively, a disperser (also referred to as the prover later) commits to $\vec{m}$ and must later prove, for each share $f(\omega^i)$, that it is consistent with the committed message. This primitive enables succinct, verifiable encoding of data across many users and underpins constructions such as verifiable secret sharing (VSS) [5], [17], [18] and blockchain data availability (DA) [16], [19], [20].

Given that RS encoding is naturally defined by polynomial evaluation, a straightforward approach to realizing an RS code commitment is through a *polynomial commitment scheme* (PCS) [21]. A PCS allows a prover to commit to a polynomial $f(X)$ of bounded degree such that, later, the prover can efficiently open the commitment at any evaluation point $x \in \mathbb{F}$ with a short proof π, convincing a verifier that $f(x) = y$. In the context of RS code commitment, the disperser can commit to a polynomial f of degree $k-1$ and produces n evaluation proofs showing the evaluation of $f(\omega^i)$ for all $i \in [n]$. However, this naive design is prohibitively expensive: producing n proofs for a degree-$(k-1)$ polynomial requires $O(kn)$ time.

Given the central role of RS code commitments, a substantial line of work has sought faster proof generation procedures. A bunch of works such as AMT [22], FK23 [23], ZXH+22 [17] and PolyFRIM [18] achieved $O(n \log n)$ prover time. More recently, HydraProofs [24] attained asymptotically optimal $O(n)$ proof generation by introducing an $\mathsf{OpenAll}$ algorithm for *multilinear polynomial commitment* within the GKR framework [25]. Concretely, the prover commits to both the message $\vec{m}$ and the codeword $\vec{c}$ in the form of a multilinear polynomial, proves an FFT relation [26] $\vec{c} = \mathsf{FFT}(\vec{m})$, and invokes

2026 IEEE Symposium on Security and Privacy (SP) | 979-8-3315-6065-2/26/$31.00 ©2026 IEEE | DOI: 10.1109/SP63933.2026.00190

© 2026, Yanpei Guo. Under license to IEEE.

DOI 10.1109/SP63933.2026.00190
Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.

an OpenAll algorithm to generate proofs of $\vec{c}[i]$ for the i-th verifier. While this achieves linear-time dispersal, the construction remains concretely inefficient: committing to both $\vec{m}$ and $\vec{c}$ doubles the commitment overhead. In addition, the OpenAll algorithm of HydraProofs [24] requires the PCS to be *homomorphic*, ruling out lightweight hash-based commitments that are often preferable in practice.

A different line of work by Hall-Andersen et al. observes the connection between RS code commitments and *interactive oracle proofs of proximity* (IOPPs). Their system, FRIDA [16], leverages the FRI protocol [27] to achieve a very fast prover. However, FRIDA inherits the high communication overhead of FRI: each per-node proof is $O(s \log^2 k)$ in size, where $s = \Theta(\lambda)$ denotes a query constant typically larger than 100 to achieve 100 bit security[1]. This results in proofs of roughly 500 KB, about two orders of magnitude larger than HydraProofs [24], rendering the scheme impractical for VSS or blockchain data-availability systems with numerous participants.

These two directions—PCS-based (HydraProofs) and IOPP-based (FRIDA)—highlight a fundamental trade-off in RS code commitments. PCS-based approaches achieve low communication cost but depend on homomorphic commitments and incur redundant commitment overhead, while IOPP-based approaches eliminate such requirements but suffer from large communication costs. Both limitations hinder scalable and practical deployment of verifiable data-dispersal protocols.

In this work, we revisit the design of RS code commitments from first principles, aiming to close this efficiency gap. We ask:

> *Can we construct an RS code commitment that preserves linear-time proof generation, supports any PCS, and avoids redundant commitments?*

1.1. Our Contributions

We give an affirmative answer to the above question by presenting UltraProofs, a new framework for efficient and flexible RS code commitments without redundant commitment. UltraProofs achieves asymptotically optimal $O(n)$ proof generation time. It can be integrated with any multilinear polynomial commitment scheme, while removing the need for homomorphic polynomial commitments and substantially improving concrete communication efficiency. Our contributions are summarized below.

- **UltraProofs: a PCS-agnostic RS code commitment**: We introduce UltraProofs, the first RS code commitment scheme that is fully compatible with any multilinear polynomial commitment scheme. At its core is a new technique called *evaluation consolidation*, which aggregates n evaluation proofs across RS evaluation points into a single randomized evaluation. Unlike HydraProofs [24], UltraProofs only commits to the message, reducing redundant prover work of committing codeword. It also eliminates the dependency on homomorphic commitments. This yields a strictly simpler and more modular construction that can be instantiated with a variety of PCS designs.
- **LightLigero: a new RS code–based multilinear PCS**: We further design a tailored multilinear PCS, LightLigero, by combining RS code based PCS with group based PCS. LightLigero achieves $O(n \log n) \cdot \mathbb{F} + O(\frac{n}{\log n}) \cdot \mathbb{G}$ prover time and $O(\lambda \log n)$ proof size, improving asymptotically over previous RS-based PCS schemes such as Deepfold [28] and WHIR [29]. It serves as an efficient backend for UltraProofs but is also of independent interest for general proof systems.
- **Implementation and evaluation**: We implemented UltraProofs and LightLigero in Rust using the Arkworks library [31] over BN254. Our code is open-sourced at https://github.com/guo-yanpei/ultraproofs. Across both VSS and DA benchmarks, UltraProofs demonstrates strong concrete efficiency: in VSS, it achieves a $2.4\times$ faster prover and $4\times$ smaller proofs than HydraProofs; in DA, the combination of UltraProofs and LightLigero reduces per-node communication by $2\sim5\times$ compared to FRIDA with comparable prover time. Our implementation validates that UltraProofs is not only asymptotically optimal but also practical for deployment in large-scale distributed systems.

1.2. Related Works

TABLE 1: Complexities of multilinear PCS of size n to achieve λ-bit security

Scheme	$\mathcal{P}$ time	Proof size / $\mathcal{V}$ time
PST [32]	$n \cdot \mathbb{G}$	$\log n$
Dory [33]	$n \cdot \mathbb{G}$	$\log n$
Hyrax [34]	$n \cdot \mathbb{G}$	$\sqrt{n}$
Ligero++ [35]	$n \log n \cdot \mathbb{F} + n \cdot \mathbb{H}$	$\lambda \log^2 n$
Deepfold [28]	$n \log n \cdot \mathbb{F} + n \cdot \mathbb{H}$	$\lambda \log^2 n$
WHIR [29]	$n \log n \cdot \mathbb{F} + n \cdot \mathbb{H}$	$\lambda \log n \log \log n$
LightLigero	$n \log n \cdot \mathbb{F} + n \cdot \mathbb{H} + \frac{n}{\log n} \cdot \mathbb{G}$	$\lambda \log n$

• For simplicity, the asymptotic notation $O(\cdot)$ is omitted.

Polynomial Commitment Scheme. Multilinear PCS is fundamental cryptographic primitive with extensive applications [17], [36], [37]. In Table 1, we summarize the asymptotic properties of related schemes for a comparison.

PCS can be broadly categorized into two types: group-bas ed and hash-based schemes. Group-based schemes, such as PST [32] and Dory [33], typically have very short proof sizes—concretely, only a few kilobytes. However, their prover time is usually larger, as they require $O(n)$ group multi-scalar multiplications during the commitment phase.

Hash-based schemes [28], [35], [38], [39] typically rely on error-correcting codes. These schemes use codes such as Reed–Solomon (RS) codes to first encode the polynomial and then construct a Merkle tree to commit to it. The verifier can ensure that the committed vector approximates a valid

1. Unlike FRI-based PCS [28], [29], FRIDA [16] can only work under *unique decoding radius* setting, resulting in more than λ samples to achieve λ-bit security [30].

Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.

codeword by checking a constant number of randomly sampled positions. Their concrete prover time is significantly faster, as both encoding and hashing are much cheaper than group operations. The downside is that existing hash-based schemes incur at least $O(s \log n \log\log n)$ proof size [29], where $s = \Theta(\lambda)$ is the number of queries, typically ranging from tens to thousands depending on the specific error-correcting code used. Concretely, this will be translated into at least 100KB proof size.

RS code commitment. As discussed earlier, existing RS code commitment schemes can be broadly categorized into two types: *PCS-based* and *IOPP-based* constructions.

Since RS encoding is defined by polynomial evaluation, most schemes naturally build upon a polynomial commitment. The high-level idea is straightforward: the disperser commits to a degree-$(k-1)$ polynomial $f(X)$ using a PCS and generates n evaluation proofs corresponding to $f(\omega^0), \ldots, f(\omega^{n-1})$. The core technical challenge for PCS-based schemes is to generate these n proofs faster than $O(n^2)$.

Early solutions [23] relied exclusively on the KZG commitment scheme, which supports producing n evaluation proofs in $O(n \log n)$ group exponentiation. ADVZ [9] extended this design to the *interleaved* setting (dispersing multiple RS codewords concurrently), achieving prover complexity $O(n \log n + Ln) \cdot \mathbb{G}$ for L interleaved codewords rather than a naive L-multiplicative cost. These KZG-based schemes also suffer from the weakness of trusted setup. Subsequent works such as ZXH+22 [17] and PolyFRIM [18] generalized the approach to other PCS constructions. These methods remove the need for $O(n \log n)$ group exponentiations but still incur $O(n \log n)$ hash during proof generations, leaving the prover cost dominated by these hashing operations.

HydraProofs [24] is the first PCS-based RS code commitment achieving linear prover time. It introduces an $\mathsf{OpenAll}$ algorithm [40] for multilinear PCS, enabling the simultaneous opening of all evaluations of multilinear polynomial over boolean hypercube. However, the design relies on a *homomorphic* PCS, which limits instantiations to slower, group-based schemes. Moreover, its proof generation still requires $O(n)$ group operations due to the need to commit the codeword $\vec{c}$, effectively doubling the prover's cost.

IOPP-based solutions are proposed by Hall-Andersen et al. in [15], and afterwards improved in FRIDA [16]. These schemes don't explicitly invoke a polynomial commitment scheme, but directly commit to the entire codeword through Merkle tree. The challenge here is how to guarantee a single element is consistent with the entire codeword. To circumvent this, FRIDA proves that FRI [27] can satisfy an additional consistency criterion. The major weakness of FRIDA is its $O(s \log^2(nL))$ proof size compared to other schemes. This could be prohibitive when n gets larger.

We summarize the asymptotic properties of existing RS code commitments in Table 2. In the table, prover time is split into *commit*, representing the time to generate the message to be broadcast, and *prove all*, representing the time to generate all proofs. Compared with HydraProofs [24], UltraProofs achieves concretely better prover time due to a reduction in the *prove all* cost. When combined with Dory, UltraProofs achieves better performance across all dimensions.

Moreover, UltraProofs is PCS-agnostic, meaning it can be integrated with any multilinear PCS. When paired with LightLigero, UltraProofs can achieve asymptotically faster performance than HydraProofs, at the cost of a larger proof size. However, this communication overhead becomes less significant when L is large and the data share size is already substantial. Compared with FRIDA, the proof size of UltraProofs combined with LightLigero is reduced by a factor of $\log n$, at the cost of introducing an additional sublinear MSM.

2. Preliminaries

Notations. Let $\mathbb{F}$ denote a FFT-friendly prime field. For a vector $\vec{v} \in \mathbb{F}^n$, we write $v[i]$ for its i-th entry (indices start from 0 by default). For a matrix $M \in \mathbb{F}^{m \times n}$, $M[i,j]$ denotes the element in the i-th row and j-th column, $M[i,:]$ denotes the i-th row. Given two vectors $\vec{a}, \vec{b} \in \mathbb{F}^n$, we use $\vec{a} \odot \vec{b}$ for their Hadamard (element-wise) product, and $\vec{a} \otimes \vec{b}$ for their tensor product. We denote $[n] := \{0, \ldots, n-1\}$. Given scalars $v_1, v_2, \cdots, v_n$, we use $\vec{v}_{[:i]}$ to denote subvector $(v_1, \cdots, v_i)$.

Merkle Tree. A Merkle tree provides an authenticated commitment to a vector by organizing its elements in a hash-based binary tree.

- $\mathsf{mt} \leftarrow \mathsf{MT.Construct}(\vec{v})$: Given a vector $\vec{v}$, construct a Merkle tree mt whose leaves correspond to the entries of $\vec{v}$. The root of mt, denoted as $\mathsf{mt.Root}$, can be treated as the commitment to $\vec{v}$.
- $(v_i, \mathsf{path}_i) \leftarrow \mathsf{MT.Open}(i, \mathsf{mt})$: Given index i and Merkle tree mt, it returns the i-th leaf v_i together with its authentication path path_i.
- $\mathsf{rt} \leftarrow \mathsf{MT.Recover}(\mathsf{path}_i, v_i, i)$: Given index i, the i-th leaf v_i and its corresponding path path_i, recompute the Merkle root of original Merkle tree. A verifier checks whether $\mathsf{rt} = \mathsf{mt.Root}$ to confirm consistency with the commitment.

Merkle trees are typically instantiated with collision-resistant hash functions, ensuring integrity and succinct membership proofs.

2.1. Multilinear Extension and Twin Polynomials

Lemma 1 (Multilinear extension)**.** *A multilinear polynomial is a multivariate polynomial in which the degree of each variable is at most one. For every function $f : \{0,1\}^\mu \to \mathbb{F}$, there is a unique multilinear polynomial $\tilde{f} \in \mathbb{F}^{\leq 1}[X_1, \cdots, X_\mu]$ such that $\tilde{f}(\vec{b}) = f(\vec{b})$ for all $\vec{b} \in \{0,1\}^\mu$. We call $\tilde{f}$ the multilinear extension (MLE) of f, and $\tilde{f}$ can be expressed as*

$$\tilde{f}(\vec{X}) = \sum_{\vec{b} \in \{0,1\}^\mu} f(\vec{b}) \cdot \tilde{eq}_{\vec{X}}(\vec{b})$$

Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.

TABLE 2: Complexities comparison of interleaved RS code $\mathsf{RS}^{\equiv L}[k,n]$ commitment to achieve λ-bit security

Scheme	Commit	Prove All	Proof Size	Verify	Trans.?
KZG [23]	$nL \cdot \mathbb{G}$	$nL \log n \cdot \mathbb{G}$	L	$L \cdot \mathbb{G}_p$	no
ADVZ [9]	$nL \cdot \mathbb{G}$	$n \log n \cdot \mathbb{G} + nL \cdot \mathbb{F}$	$\log n + L$	$1 \cdot \mathbb{G}_p + \log n \cdot \mathbb{H}$	no
ZXH+22 + Dory [17]	$nL \cdot \mathbb{G}$	$nL \cdot \mathbb{G} + nL \log n \cdot \mathbb{H}$	$L \cdot \log^2 n$	$L \log^2 n \cdot \mathbb{H}$	yes
ZXH+22 + Virgo [17]	$nL \log n \cdot \mathbb{F} + nL \cdot \mathbb{H}$	$nL \log n \cdot \mathbb{H}$	$\lambda L \cdot \log^2 n$	$\lambda L \log^2 n \cdot \mathbb{H}$	yes
PolyFRIM [18]	$nL \log n \cdot \mathbb{F} + nL \cdot \mathbb{H}$	$nL \log n \cdot \mathbb{H}$	$\lambda L \cdot \log^2 n$	$\lambda L \log^2 n \cdot \mathbb{H}$	yes
FRIDA [16]	$nL \log n \cdot \mathbb{F} + nL \cdot \mathbb{H}$	$nL \cdot \mathbb{H}$	$\lambda \cdot \log^2(nL)$	$\lambda \cdot \log^2(nL) \cdot \mathbb{H}$	yes
HydraProofs + Dory	$nL \cdot \mathbb{G}$	$nL \cdot \mathbb{G}$	$\log^2 n$	$\log^2 n \cdot \mathbb{H} + \log(nL) \cdot \mathbb{G}$	yes
UltraProofs + Dory	$nL \cdot \mathbb{G}$	$nL \cdot \mathbb{H}$	$\frac{\log^2 n}{\log\log n}$	$\frac{\log^2 n}{\log\log n} \cdot \mathbb{H} + \log(nL) \cdot \mathbb{G}$	yes
UltraProofs + LightLigero	$nL \log n \cdot \mathbb{F} + nL \cdot \mathbb{H}$	$nL \cdot \mathbb{H} + \frac{nL}{\log(nL)} \cdot \mathbb{G}$	$\lambda \log(nL)$	$\lambda \log(nL) \cdot \mathbb{H} + \log(nL) \cdot \mathbb{G}$	yes

- The asymptotic notation $O(\cdot)$ in all terms is omitted.
- $\mathbb{G}$ denotes group exponentiation, $\mathbb{H}$ denotes Hash operation, $\mathbb{F}$ denotes field multiplication with decreasing unit operation cost $\mathbb{G} > \mathbb{H} > \mathbb{F}$.
- For schemes do not support interleaved RS code commitment we multiply L for every term of cost.

where $\tilde{eq}_{\vec{X}}(\vec{b}) = \prod_{i\in[\mu]} \left(\vec{b}[i]\vec{X}[i] + (1-\vec{b}[i])(1-\vec{X}[i])\right)$.

Lemma 2 (Twin Polynomials)**.** *Let $\tilde{f}(X_1,\ldots,X_\mu)$ be a μ-variate multilinear polynomial. We define a univariate polynomial $f(X)$ that shares the same coefficient vector $\vec{f} \in \mathbb{F}^{2^\mu}$ with $\tilde{f}$, such that:*

$$\tilde{f}(x_1,\ldots,x_\mu) = \langle \vec{f}, (1,x_1) \otimes \cdots \otimes (1,x_\mu)\rangle$$
$$f(x) = \langle \vec{f}, (1, x, x^2, \ldots, x^{2^\mu-1})\rangle$$

Then $\tilde{f}$ and f are said to be twin polynomials*, satisfying*

$$f(x) = \tilde{f}(x^{2^0}, x^{2^1}, \ldots, x^{2^{\mu-1}})$$

We extend this notion to bivariate polynomials. Let $f(X,Y) \in \mathbb{F}[X,Y]$ have degree $2^\ell - 1$ in X and $2^\kappa - 1$ in Y, and let $\tilde{f}$ be a $(\ell+\kappa)$-variate multilinear polynomial sharing coefficient vector $\vec{f} \in \mathbb{F}^{2^{\ell+\kappa}}$. Then f and $\tilde{f}$ are twin polynomials if for all $x, y \in \mathbb{F}$ and $(x_1,\ldots,x_{\ell+\kappa}) \in \mathbb{F}^{\ell+\kappa}$:

$$f(x,y) = \langle \vec{f}, (1, x, x^2, \ldots, x^{2^{\ell-1}}) \otimes (1, y, y^2, \ldots, y^{2^{\kappa-1}})\rangle$$
$$\tilde{f}(x_1,\ldots,x_{\ell+\kappa}) = \langle \vec{f}, (1,x_1) \otimes \cdots \otimes (1,x_{\ell+\kappa})\rangle.$$

2.2. Reed–Solomon Codes

Reed–Solomon Code. A Reed–Solomon (RS) code [41] encodes a message as evaluations of a low-degree polynomial. Given an evaluation set $E = (e_1,\ldots,e_n) \subseteq \mathbb{F}$, the RS code $\mathsf{RS}[k,n,\mathbb{F}] : \mathbb{F}^k \to \mathbb{F}^n$ encodes $\vec{m} \in \mathbb{F}^k$ as follows: Interpret $\vec{m}$ as polynomial $f(X)$ of degree smaller than k, whose coefficients are $\vec{m}$. Output codeword

$$\vec{c} = (f(e_1),\ldots,f(e_n)).$$

Typically, E is chosen to be a multiplicative subgroup $\Omega = (1,\omega,\omega^2,\cdots,\omega^{n-1})$, where ω is the n-th root of unity, so that the encoding can be accelerated via FFT. The j-th symbol of codeword $\vec{c}$ is $f(\omega^j)$, $j \in [n]$.

Interleaved RS Codes. Given $\mathsf{RS}[k,n,\mathbb{F}]$, define an interleaved code $\mathsf{RS}^{\equiv L}[k,n] : \mathbb{F}^{L\times k} \to \mathbb{F}^{L\times n}$. Encoding M applies RS independently to each row $M[i,:]$, yielding $C[i] \in \mathbb{F}^n$. The j-th encoded symbol is then $(C[0,j],\ldots,C[L-1,j]) \in \mathbb{F}^L$. Interleaving preserves the decoding threshold of the base code.

2.3. Reed–Solomon Code Commitment

A Reed–Solomon code commitment [15] allows committing to an interleaved RS codeword $c \in \mathsf{RS}^{\equiv L}$.

Definition 1 (Erasure Code Commitment)**.** *Let $\mathsf{RS}^{\equiv L} : \mathbb{F}^{L\times k} \to (\mathbb{F}^L)^n$. An erasure code commitment scheme CC is a tuple $(\mathsf{Setup}, \mathsf{Commit}, \mathsf{Open}, \mathsf{Verify})$ of PPT algorithms:*

- $\mathsf{Setup}(1^\lambda) \to \mathsf{gp}$*: generate global parameters.*
- $\mathsf{Commit}(\mathsf{gp}, M) \to (\mathsf{com}, St)$*: commit to message $M \in \mathbb{F}^{L\times k}$, producing a commitment com and state St.*
- $\mathsf{Prove}(\mathsf{gp}, St, i) \to \pi$*: produce an opening proof for position $i \in [n]$.*
- $\mathsf{Verify}(\mathsf{gp}, \mathsf{com}, i, \hat{m}_i, \pi) \to \{0,1\}$*: verify that $\hat{m}_i$ corresponds to index i of the committed codeword.*

An RS code commitment must satisfy *completeness*, *position binding*, and *code binding* (see Appendix A.1). Intuitively, RS code commitments are binding vector commitment with the additional property that any set of openings is consistent with a RS codeword.

2.4. Polynomial Commitments

Definition 2 (Multilinear Polynomial Commitment)**.** *A multilinear polynomial commitment PC is a tuple $(\mathsf{Setup}, \mathsf{Commit}, \mathsf{OpenPoly}, \mathsf{Eval}, \mathsf{Verify})$:*

- $\mathsf{Setup}(1^\lambda, \mu) \to \mathsf{pp}$*: generate public parameters for μ-variate polynomials.*
- $\mathsf{Commit}(\mathsf{pp}, \tilde{f}) \to (\mathcal{C}, \mathcal{D})$*: commit to a multilinear polynomial $\tilde{f}$, outputting commitment $\mathcal{C}$ and auxiliary data $\mathcal{D}$.*
- $\mathsf{OpenPoly}(\mathsf{pp}, \mathcal{C}, \tilde{f}, \mathcal{D}) \to \{0,1\}$*: verify consistency between $\tilde{f}$ and its commitment.*
- $\mathsf{Eval}(\mathsf{pp}, \mathcal{D}, \vec{z}, \tilde{f}) \to (y, \pi)$*: output evaluation result $y = \tilde{f}(\vec{z})$ and proof π.*
- $\mathsf{Verify}(\mathsf{pp}, \mathcal{C}, \vec{z}, y, \pi) \to \{0,1\}$*: verify that y is a correct evaluation of the committed polynomial.*

A valid PCS must satisfy completeness, binding, and (knowledge) soundness, defined in Appendix A.2.

Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.

Sumcheck. Given a commitment to multilinear polynomials $\tilde{f}_1, \cdots, \tilde{f}_d$ and a public scalar $y \in \mathbb{F}$, verifier can be convinced that

$$\sum_{\vec{b}} \left(\prod_{i=1}^{d} \tilde{f}_i(\vec{b}) \right) = y$$

through classic sumcheck protocol. The protocol is presented in Appendix B.

Partial Evaluation. Partial evaluation of a bivariate polynomial $f(X, Y)$ is defined as evaluating $v(X) = f(X, y)$ for some $y \in \mathbb{F}$. To prove that $v(X)$ is correctly derived from $f(X, Y)$, it suffices to show $f(\beta, y) = v(\beta)$ for a random $\beta \in \mathbb{F}$ sampled by the verifier.

Batch Evaluation. Batched evaluation of PCS proves multiple evaluations jointly:

$$\mathsf{BatchEval}(\mathsf{pp}, \tilde{f}, \{\vec{z}^{(i)}\}_{i\in[m]}) \to (\{y_i\}_{i\in[m]}, \pi),$$
$$\mathsf{Verify}(\mathsf{pp}, \mathcal{C}, \{\vec{z}^{(i)}\}, \{y_i\}, \pi) \to \{0, 1\}.$$

This can be realized via the *sumcheck* protocol (Appendix B):

$$\sum_{\vec{b}\in\{0,1\}^{\mu}} \tilde{f}(\vec{b}) \cdot \left(\sum_{i\in[m]} \alpha^i \cdot \tilde{eq}_{\vec{z}^{(i)}}(\vec{b}) \right) \stackrel{?}{=} \sum_{i\in[m]} \alpha^i y_i,$$

for random $\alpha \in \mathbb{F}$. Prover time is $O(mn)$, proof size is $O(m + \log n)$ and verifier time is $O(m \log n)$.

Multi-Evaluation. Unlike batched evaluation (shared proof), *multi-evaluation* produces separate proofs for multiple queries:

$$\mathsf{MultiEval}(\mathsf{pp}, \tilde{f}, \{\vec{z}^{(i)}\}) \to \{(y_i, \pi_i)\}$$
$$\mathsf{Verify}(\mathsf{pp}, \mathcal{C}, \vec{z}^{(i)}, y_i, \pi_i) \to \{0, 1\}$$

Multi-evaluation favors independent verification and reduced prover complexity.

Multi-Partial Evaluation. This hybrid variant proves the correctness of multiple partial evaluations. For $f(X, Y)$ and $v_i(X) = f(X, y_i)$:

$$\mathsf{MultiPartialEval}(\mathsf{pp}, f, \{y_i\}) \to \{(v_i(X), \pi_i)\}$$
$$\mathsf{Verify}(\mathsf{pp}, \mathcal{C}, y_i, v_i(X), \pi_i) \to \{0, 1\}$$

Connection to RS Code Commitment. Since RS codes are evaluations of univariate polynomials, a multi-evaluation of PCS naturally realizes an RS code commitment. Given $\mathsf{RS}[k, n]$ over domain $\Omega \subset \mathbb{F}$, committing to $f(X)$ (degree $< k$) implicitly commits to the codeword $(f(x))_{x\in\Omega}$. Each symbol proof corresponds to one PCS evaluation, establishing a tight algebraic connection between polynomial commitments and RS code commitments.

3. Technical Overview

In this section, we provide an overview of our Reed–Solomon code commitment scheme. Section 3.1 presents *UltraProofs*, a general RS code commitment compatible with any multilinear PCS. The dominant prover cost of UltraProofs is invoking a multilinear PCS to commit the original message; in contrast, HydraProofs requires committing both the message and its encoded codeword. Section 3.2 then introduces LightLigero, a new multilinear PCS specifically tailored for interleaved RS code commitments, which achieves substantially smaller proof sizes than existing RS code–based PCS.

3.1. UltraProofs: Reed–Solomon Code Commitment

UltraProofs eliminates the redundant polynomial commitment present in prior RS code commitments while preserving linear-time proof generation. Our construction builds on the structural connection between RS codes and polynomials. As discussed in Section 2.4, a natural baseline is to realize RS code commitments through multi-evaluation of a univariate PCS: the prover commits to a degree-$(k-1)$ polynomial $f(X)$ and generates n proofs for the evaluations $\{f(\omega^i)\}_{i\in[n]}$, where ω is an n-th root of unity. However, efficiently producing n proofs for a generic univariate PCS remains challenging.

From univariate to multilinear representation. Instead of committing to $f(X)$ directly, we observe that the prover can commit to its *twin polynomial* $\tilde{f}$, defined in Lemma 2. Evaluating $f(\omega^i)$ is equivalent to evaluating the multilinear polynomial

$$\tilde{f}(\omega^i, \omega^{2i}, \ldots, \omega^{2^{\mu-1}\cdot i}),$$

where $\mu = \log n$. Thus, it suffices to generate n evaluation proofs for $\tilde{f}$ instead of f, enabling the use of multilinear PCS machinery.

Evaluation reduction: fixed to random. A key advantage of multilinear PCS is the existence of *evaluation reduction* protocols, which allow the prover to reduce evaluation at an arbitrary point $\vec{z}$ to evaluation at a random point $\vec{r}$. For example, when proving $\tilde{f}(\vec{z}) = y$, $\mathcal{P}$ can first use a lightweight sumcheck to prove

$$\sum_{\vec{b}} \tilde{f}(\vec{b}) \cdot \tilde{eq}_{\vec{z}}(\vec{b}) = y$$

which reduces to a claim about a random evaluation $\tilde{f}(\vec{r}) = y_r$. Using this technique, the disperser can reduce n distinct evaluations into a single random evaluation $\tilde{f}(\vec{r})$, and invoke PC.Eval to prove the reduced evaluation. A significant advantage of this RS code commitment paradigm is that it commits only to the polynomial $\tilde{f}$, of the same size as the message. It avoids redundant committing to the codeword, as in HydraProofs [24].

However, as the multiplier $\tilde{eq}_{\vec{z}}(\vec{b})$ depends on all coordinates of $\vec{z}$, the concurrent n sumcheck protocols require distinct messages. As a result, the prover must execute n independent sumcheck instances, each with $\log n$ rounds and point-dependent messages, leading to an overall at least $O(n \log n)$ computational and communication cost.

Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.

To overcome this bottleneck, we design a new evaluation reduction protocol that avoids the dependence on the entire $\vec{z}$. In each round $i \in [1, \mu]$, the prover sends a linear function

$$l_i(X) := \tilde{f}(r_1, \cdots, r_{i-1}, X, \vec{z}[i+1:]),$$

and the verifier checks consistency $l_i(z_i) = l_{i-1}(r_{i-1})$[2], samples a new random r_i, and continues. After μ rounds, the verifier only needs to check $\tilde{f}(\vec{r}) = l_\mu(r_\mu)$ using the underlying PCS.

Soundness follows an argument analogous to the sum-check protocol [42]: if $l_{i-1}(r_{i-1})$ is incorrect, then so is $l_i(X)$ for all but a negligible fraction of random r_i. Hence, an incorrect claim about $\tilde{f}(\vec{z})$ implies an incorrect claim about $\tilde{f}(\vec{r})$ with overwhelming probability.

Our evaluation reduction is *PCS-agnostic*—it operates purely at the level of evaluation relations without depending on specific commitment or evaluation algorithms. This agnosticism allows flexible instantiation with any multilinear PCS, unlike prior techniques that are tightly coupled to IOPP-based [16] or group-based commitments [14], [43].

Evaluation consolidation: many fixed to single random. To achieve efficient multi-evaluation, we extend evaluation reduction to handle many points concurrently. The key observation is that if all reductions share the same verifier randomness $\vec{r}$, then after each round of reduction, the number of distinct evaluation claims halves. For example, for evaluation points $(\omega^i, \omega^{2i}, \ldots)$ and $(\omega^{i+n/2}, \omega^{2i}, \ldots)$, the two claims merge after the first randomization step since they differ only in the first variable. Thus, after μ rounds, all claims consolidate into the single point $\vec{r}$.

To ensure a shared random challenge across all concurrent reductions, we employ a modified Fiat–Shamir transformation following [17]. In each round, the prover constructs a Merkle tree over all per-instance randomness and derives the shared challenge from the Merkle root. Each verifier is provided with its corresponding Merkle path as proof of correct challenge derivation. Because the number of distinct messages halves per round, the total hashing cost of this procedure is $O(n)$. The proof size is $O(\log^2 n)$, as there are $\log n$ rounds, and each round includes a Merkle path with an amortized length of $O(\log n)$.

Reducing communication overhead. We further reduce round complexity—and thus communication—by batching variable randomization. Instead of randomizing a single variable per round, the prover can send an s-variate polynomial $l_i(X_1, \ldots, X_s)$, thereby randomizing $s > 1$ variables at once. This reduces the number of rounds from μ to μ/s while slightly increasing soundness error from $\mu/|\mathbb{F}|$ to $(\mu/s) \cdot (2^s/|\mathbb{F}|)$, which remains negligible for $s = O(\log \log n)$. The total proof size becomes $O(\log^2 n / \log \log n)$.

Interleaved RS code commitment. Combining commitment to $\tilde{f}$, evaluation consolidation, and the final evaluation of $\tilde{f}(\vec{r})$ yields a complete multi-evaluation scheme and hence an RS code commitment. Extending from a single-layer to an interleaved RS code $\mathsf{RS}^{\equiv L}$, one might naively run L separate consolidations, leading to L-fold communication overhead.

We avoid this blow-up by viewing the entire message $M \in \mathbb{F}^{L \times k}$ as a *bivariate* polynomial $f(X, Y)$ with $\deg_X(f) < k$ and $\deg_Y(f) < L$. The i-th verifier partially evaluates $f(X, Y)$ at $Y = \omega^i$ to obtain $v_i(X) = f(X, \omega^i)$, and checks that $v_i(\beta) = f(\beta, \omega^i)$ for a random $\beta \in \mathbb{F}$. The evaluation claims $\{f(\beta, \omega^i)\}_{i \in [n]}$ can be consolidated using our technique. Therefore, the prover only needs to commit to the twin polynomial $\tilde{f}$ of $f(X, Y)$, and finally evaluates $\tilde{f}(\vec{\beta}, \vec{r})$ after evaluation consolidation, where $\vec{\beta} = (\beta, \beta^2, \ldots, \beta^{L/2})$. A complete construction is presented in Section 4.

3.2. LightLigero: Multilinear PCS with $O(\lambda \log n)$ Proof Size

A multilinear PCS is a critical component of UltraProofs. Existing PCS schemes impose significant overhead in either computation or communication. We design a new PCS optimized for the cost profile required in interleaved RS code commitment, especially data-availability (DA) applications.

Polynomial commitment schemes can be roughly divided into two categories: group-based and code-based. Group-based PCS (e.g., PST [32], Dory [33]) achieve compact proofs but require expensive multi-scalar multiplications (MSM). Code-based PCS (e.g., Deepfold [28]) rely on Merkle-tree commitments, offering fast prover time at the cost of larger proof size: committing to $O(\log n)$ trees, each opening $\Theta(\lambda)$ paths, yields $O(\lambda \log^2 n)$ proof size. For example, for a polynomial of size 2^{20}, PST produces a $\sim$1 KB proof in $\sim$7 s, whereas Deepfold achieves $\sim$2 s prover time with a 300 KB proof (assuming code rate $1/4$).

Our PCS combines the advantages of both approaches. Inspired by Ligero++ [35], the prover commits to the multilinear polynomial $\tilde{f}$ via an *interleaved RS code*. Specifically, the polynomial coefficients are arranged into a $\log n \times (n/\log n)$ matrix C, which is encoded under $\mathsf{RS}^{\equiv \log n}$. Each column of C is committed as a leaf in a Merkle tree, forming the public commitment. Let $\mu' = \log \log n$. During evaluation at $(z_1, \ldots, z_\mu)$, the prover commits to the reduced $(\mu - \mu')$-variate polynomial

$$\tilde{f}'(X_{\mu'+1}, \ldots, X_\mu) := \tilde{f}(z_1, \ldots, z_{\mu'}, X_{\mu'+1}, \ldots, X_\mu)$$

using a group-based PCS (e.g., Dory [33]) and proves its evaluation $\tilde{f}'(z_{\mu'+1}, \ldots, z_\mu)$. The central challenge is to prove consistency between the RS-code-committed polynomial $\tilde{f}$ and the group-committed $\tilde{f}'$.

We ensure consistency by sampling λ random columns from the interleaved RS code and performing batch evaluations of $\tilde{f}'$ at λ corresponding points. The overall prover time, including both commitment and evaluation, is

$$O\Big(n \log n \cdot \mathbb{F} + n \cdot \mathbb{H} + \tfrac{n}{\log n} \cdot \mathbb{G}\Big),$$

where $\mathbb{F}$, $\mathbb{H}$, and $\mathbb{G}$ denote the costs of field operations, hashes, and group operations, respectively. The proof size is

2. We set $l_0(\cdot) = \tilde{f}(\vec{z}) = y$.

Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.

$O(\lambda \log n)$, as it involves opening λ columns of the Merkle-committed matrix C.

This hybrid PCS achieves roughly 2.5 s prover time and ∼50 KB proof size for $n = 2^{20}$, maintaining the efficiency of code-based schemes while tremendously reducing the proof size.

4. Ultraproofs: Reed–Solomon Code Commitment

This section formalizes our interleaved Reed–Solomon code commitment scheme, which realizes multi-partial evaluation for any multilinear polynomial commitment scheme. The construction hinges on a new primitive called the *evaluation reduction* protocol, which forms the foundation for efficient, concurrent proof generation.

4.1. Evaluation Reduction

Given a commitment to a multilinear polynomial $\tilde{f}(X_1, \ldots, X_\mu)$, the verifier $\mathcal{V}$ wishes to verify that a claimed value y satisfies $\tilde{f}(\vec{z}) = y$. Our evaluation reduction protocol enables $\mathcal{V}$ to interact with the prover $\mathcal{P}$ to reduce this claim to an equivalent claim on a random point $\vec{r} \in \mathbb{F}^\mu$, namely $\tilde{f}(\vec{r}) = y_r$. We denote this process as *evaluation reduction*, with its formal definition deferred to Appendix C.

A classical way to achieve such a reduction is to employ a sumcheck protocol to prove

$$\sum_{\vec{b} \in \{0,1\}^\mu} \tilde{f}(\vec{b}) \cdot \tilde{eq}_{\vec{z}}(\vec{b}) = y.$$

At the final step, the verifier checks the claim on $\tilde{f}(\vec{r}) \cdot \tilde{eq}_{\vec{z}}(\vec{r})$, where vector $\vec{r}$ is the concatenation of random challenge used in each round. Because $\tilde{eq}_{\vec{z}}(\vec{r})$ can be computed locally, the prover's task reduces to proving $\tilde{f}(\vec{r}) = y_r$ via PCS evaluation. This idea underlies prior works such as Basefold [44] and HyperNOVA [45].

Limitations of prior reduction. Despite its generality, the sumcheck-based reduction introduces two inefficiencies for our use case. First, in each round the prover must send a degree-2 polynomial

$$g_i(X) = \sum_{\vec{b}_{[i+1:]}} \tilde{f}(\vec{r}_{[:i-1]}, X, \vec{b}_{[i+1:]}) \cdot \tilde{eq}_{\vec{z}}(\vec{r}_{[:i-1]}, X, \vec{b}_{[i+1:]})$$

whose coefficients depend on the entire vector $\vec{z}$. This dependence prevents reusing messages across multiple reductions at different evaluation points, making multi-evaluation batching infeasible. Second, the quadratic degree of $g_i(X)$ leads to heavier concrete computation and larger communication compared to the linear case.

Our efficient reduction. We propose a more efficient evaluation reduction protocol that removes both inefficiencies. Instead of transmitting a quadratic polynomial $g_i(X)$, the prover sends only a linear function

$$l_i(X) = \tilde{f}(\vec{r}_{[:i-1]}, X, \vec{z}_{[i+1:]})$$

This simplification yields two concrete advantages. (1) Each round involves only sending one field element (because $l_i(z_i)$ has been fixed), reducing both verifier time and message size. (2) The message $l_i(X)$ is independent of the prefix $\vec{z}_{[:i-1]}$, enabling all reductions over distinct $\vec{z}$ to share verifier randomness—a crucial property for the evaluation consolidation introduced later.

Round reduction. Round complexity can be reduced by a factor of s by randomizing s variables at once: in each round, the prover sends a univariate polynomial $p(X)$ of degree at most $2^s - 1$, such that

$$p_i(X) = \tilde{f}(\vec{r}_{[:i-1]}, X, X^2, \ldots, X^{2^{s-1}}, \vec{z}_{[i+1:]}),$$

thereby jointly handling s coordinates per round instead of one. The formal protocol description is presented in Protocol 5, delayed in Appendix C. The following completeness and soundness proofs are provided in Appendix D.

Theorem 1 (Completeness)**.** *If $\tilde{f}(\vec{z}) = y$ and both $\mathcal{P}$ and $\mathcal{V}$ are honest, then the output satisfies $\tilde{f}(\vec{r}) = y_r$.*

Theorem 2 (Soundness)**.** *If $\tilde{f}(\vec{z}) \neq y$, then* $\Pr[\tilde{f}(\vec{r}) = y_r] \leq \frac{\mu \cdot 2^s}{s|\mathbb{F}|}$.

4.2. Non-Interactive Evaluation Consolidation

The *evaluation consolidation* protocol extends the above single evaluation reduction to n evaluations $\left\{\tilde{f}(\omega^i, \omega^{2i}, \ldots, \omega^{2^{\mu-1} \cdot i})\right\}_{i \in [n]}$ in parallel, reducing them to a single random evaluation $\tilde{f}(\vec{r})$ while producing n independent proofs. Each proof π_i reduced claim $\tilde{f}(\omega^i, \omega^{2i}, \ldots, \omega^{2^{\mu-1} \cdot i}) = y_i$ into $\tilde{f}(\vec{r}) = y_r$.

Next, we explain how to achieve linear time evaluation consolidation. The key observation is that these evaluation points have a structured form: for any i, the i-th and $(i + n/2)$-th evaluation points differ only in the first coordinate, since $\omega^{2i} = \omega^{2(i+n/2)}$. This structure enables *localized interpolation* and *per-round halving*.

Consider first the simple case where $s = 1$. For the indices i and $i + n/2$, the linear function $l(X) = \tilde{f}(X, \omega^{2i}, \ldots, \omega^{2^{\mu-1} \cdot i})$ can be interpolated from only two *existing* evaluations, namely $\tilde{f}(\omega^i, \omega^{2i}, \ldots)$ and $\tilde{f}(\omega^{i+n/2}, \omega^{2i}, \ldots)$ Hence, computing $l(X)$ for the claims $f(\omega^i)$ and $f(\omega^{i+n/2})$ requires only $O(1)$ prover time. Moreover, after substituting the first variable with r_1, the claims for $f(\omega^i)$ and $f(\omega^{i+n/2})$ merge into a single claim. As a result, the number of distinct evaluation claims is halved in each round. Therefore, the total prover time is $O(n + \frac{n}{2} + \frac{n}{4} + \cdots) = O(n)$.

This idea can be generalized to the case where $s > 1$. In this setting, $l(X)$ is a univariate polynomial of degree $2^s - 1$, defined as

$$l(X) = \tilde{f}(X, X^2, \cdots, X^{2^{s-1}}, \omega^{2^s i}, \cdots, \omega^{2^{\mu-1} i}).$$

This polynomial can be interpolated from the 2^s evaluations $\left\{f\left(\omega^{i + j \cdot \frac{n}{2^s}}\right)\right\}_{j \in [2^s]}$, since their corresponding evaluation

Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.

points share the same suffix $(\omega^{2^s i}, \cdots, \omega^{2^{\mu-1} i})$. Hence, the same localized interpolation property holds, and the number of distinct claims is divided by 2^s in each round.

Another issue is although a single evaluation reduction can be rendered non-interactive via Fiat–Shamir (FS), sharing a common random point $\vec{r}$ across all n reductions without compromising soundness can be non-trivial. We adopt the approach of [17], in which the prover constructs a Merkle tree over all n per-round messages and derives each shared challenge as the Merkle root. Each verifier receives its corresponding Merkle path as part of π_i, ensuring both soundness and independence.

The above transformation introduces an $O(\frac{\log n}{s} \cdot (\log n + 2^s))$ proof size, stemming from $\frac{\log n}{s}$ rounds, each requiring a Merkle path of length $O(\log n)$ and Merkle leaf of size $O(2^s)$. Setting $s = \log\log n$ yields a total communication complexity of $O(\log^2 n / \log\log n)$. The complete non-interactive consolidation protocol is shown in Protocol 1.

4.3. Interleaved RS Code Commitment

Combining evaluation consolidation with any multilinear PCS yields a complete RS code commitment. In the commit phase, prover invokes a PCS to commit the multilinear polynomial $\tilde{f}$ encoding the dispersed message. In proving phase, prover first consolidates all n evaluation claims to a common random point $\vec{r}$ via Protocol 1, and invokes PC.Eval to prove $\tilde{f}(\vec{r})$.

Interleaved generalization. Our scheme naturally extends to *interleaved* RS code commitments. For simplicity, assume that parameters L, k, and n in $\mathsf{RS}^{\equiv L}[k, n]$ are powers of two. We interpret the message matrix $M \in \mathbb{F}^{L \times k}$ as a bivariate polynomial $f(X, Y)$ with $\deg_X(f) < L$ and $\deg_Y(f) < k$. Each encoded symbol corresponds to a partial evaluation $v_j(X) = f(X, \omega^j)$.

Let $\ell = \log L$ and $\kappa = \log k$. There exists a multilinear polynomial $\tilde{f}$ in $\ell + \kappa$ variables such that $f(x, y) = \tilde{f}(x^{2^0}, \ldots, x^{2^{\ell-1}}, y^{2^0}, \ldots, y^{2^{\kappa-1}})$ for all $x, y \in \mathbb{F}$. The prover commits to $\tilde{f}$, samples a random β via FS, and proves the evaluation $\tilde{f}(\vec{\beta}, \vec{\omega}_j)$ for each party j, where $\vec{\beta} = (\beta, \beta^2, \ldots, \beta^{2^{\ell-1}})$ and $\vec{\omega}_j = (\omega^j, \omega^{2j}, \ldots, \omega^{2^{\mu-1} j})$. Using our consolidation protocol, all n partial claims consolidate into a single evaluation $\tilde{f}(\vec{\beta}, \vec{r})$, verified via one PCS opening. The full protocol appears in Protocol 2.

Cost analysis. UltraProofs achieves concrete efficiency compared to prior works.

- **Prover Time**: The prover's cost consists of roughly four components: committing to a multilinear polynomial of size kL, Reed–Solomon (RS) encoding of the dispersed message with complexity $O(Ln \log n \cdot \mathbb{F})$, evaluation consolidation dominated by $O(Ln \cdot \mathbb{H})$, and opening the polynomial at the consolidated point $\vec{r}$. Typically, the cost of opening in a PCS is significantly lower than that of committing [46], and evaluation consolidation is asymptotically cheaper than RS encoding. Hence, the dominant prover cost mainly lies in committing to the message via PCS and performing RS encoding. Furthermore, when using RS code-based PCS constructions such as Deepfold or LightLigero, the encoding cost can be amortized across both components.
- **Communication**: Beyond the message shares, each verifier receives the PCS commitment, the evaluation reduction proof, and the PCS evaluation proof. Thus, the total proof size equals the sum of the PCS proof size and the evaluation reduction proof size. When employing group-based PCS schemes like Dory, the communication cost is dominated by the evaluation reduction proof of size $O\left(\frac{\log^2 n}{\log\log n}\right)$. Moreover, in applications where L is large, the message can constitute the majority of the total communication cost.
- **Verifier Time**: The verification cost scales with the size of the communication asymptotically. In practice, verification completes in under 0.1 seconds, making it negligible compared to proof generation.

5. LightLigero: RS Code–Based PCS with $O(\lambda \log n)$ Proof Size

This section presents our polynomial commitment scheme LightLigero, which attains $O(n \log n) \cdot \mathbb{F} + O(n) \cdot \mathbb{H} + O(\frac{n}{\log n}) \cdot \mathbb{G}$ prover time[3] and $O(\lambda \log n)$ proof size. Concretely, the prover time is slightly larger than that of code-only schemes such as Deepfold, but the proof size is substantially smaller.

5.1. LightLigero Construction

Let $\tilde{f}(X_1, \ldots, X_\mu)$ be a μ-variate multilinear polynomial with $n = 2^\mu$ coefficients. Set $\mu' = \log\log n$. We decompose $\tilde{f}$ into $2^{\mu'}$ components by expanding on the first μ' variables:

$$\tilde{f}(X_1, \ldots, X_\mu) = \sum_{\vec{b} \in \{0,1\}^{\mu'}} \tilde{f}_{\vec{b}}(X_{\mu'+1}, \ldots, X_\mu) \cdot \prod_{i=1}^{\mu'} X_i^{\vec{b}[i]},$$

where each $\tilde{f}_{\vec{b}}$ is multilinear in $(\mu - \mu')$ variables and has size $n / \log n$.

Commit. For each $\vec{b}$, the prover encodes the coefficient vector of $\tilde{f}_{\vec{b}}$ using an *interleaved* Reed–Solomon code with rate ρ, forming a matrix of size $2^{\mu'} \times (2^{\mu-\mu'}/\rho)$ (rows index $\vec{b}$, columns index RS symbol). The prover then constructs a Merkle tree over the *columns* of this matrix and publishes the Merkle root as the commitment to $\tilde{f}$.

Evaluate. To prove $\tilde{f}(\vec{z}) = y$ for $\vec{z} = (z_1, \ldots, z_\mu)$, the prover defines the $(\mu - \mu')$-variate polynomial

$$\tilde{f}'(X_{\mu'+1}, \ldots, X_\mu) = \sum_{\vec{b} \in \{0,1\}^{\mu'}} \tilde{f}_{\vec{b}}(X_{\mu'+1}, \ldots, X_\mu) \cdot \prod_{i=1}^{\mu'} z_i^{\vec{b}[i]},$$

so that $\tilde{f}'(z_{\mu'+1}, \ldots, z_\mu) = \tilde{f}(\vec{z})$. The prover commits to $\tilde{f}'$ using a group-based PCS, e.g., Dory [32], and proves the evaluation $\tilde{f}'(z_{\mu'+1}, \cdots, z_\mu) = y$.

3. including committing and open one point

Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.

Protocol 1. Non-interactive Evaluation Consolidation NIEC

Notation: $s \in \mathbb{N}^+$ is a tunable hyperparameter that determines the degree of univariate polynomial sent in each round of the evaluation reduction. $\rho(\cdot) \to \mathbb{F}$ is a random oracle.

- $\mathsf{Prove}(\vec{c}) \to (\vec{r}, \vec{\pi})$: Given the RS code $\vec{c} \in \mathsf{RS}[k, n, \mathbb{F}]$, it outputs the consolidated evaluation point $\vec{r} \in \mathbb{F}^{\mu}$, the consolidation proofs $\vec{\pi}_j$ for $\vec{c}[j]$.
  1) Let $\omega_0 \in \mathbb{F}$ denote the n-th root of unity, $\omega_k = \omega_{k-1}^{2^s}$; Define $\mu \in \mathbb{N}$ such that $2^{\mu} = k$. Let $\vec{r}$, $\vec{mt}$ be empty vectors. $d \leftarrow \lfloor \frac{\mu - 1}{s} \rfloor$.
  2) For $k \in [1, d]$:
     a) Define $\vec{p}^{(k)}$ as a vector of length $n/2^{k \cdot s}$, such that the i-th element of $\vec{p}^{(k)}$ is a univariate polynomial of degree $2^s - 1$, interpolated from points $\left(\omega_{k-1}^{i + j \cdot n/2^{k \cdot s}}, c^{(k-1)}[i + j \cdot n/2^{k \cdot s}]\right)$ for all $j \in [2^s]$
     b) $mt^{(k)} \leftarrow \mathsf{MT.Construct}\left(\vec{p}^{(k)}\right)$, $r_k \leftarrow \rho\left(mt^{(k)}.\mathsf{Root} || r_{k-1}\right)$ be a scalar. Append $\left(r_k^{2^0}, r_k^{2^1}, \cdots, r_k^{2^{s-1}}\right)$ to $\vec{r}$.
     c) Define $\vec{c}^{(k)}$ be a vector of $n/2^{k \cdot s}$, where the i-th element of $\vec{c}^{(k)}$ is $p_i^{(k)}(r_k) \in \mathbb{F}$, where $p_i^{(k)} \in \mathbb{F}[X]$ is the i-th element of $\vec{p}^{(k)}$.
  3) Interpolate a univiriate polynomial $p^{(d+1)}(X)$ of degree $2^{\mu - d \cdot s} - 1$ from points $\left(\omega_d^i, c_i^{(d)}\right)$ for all $i \in [n/2^{d \cdot s}]$.
  4) $r_{d+1} \leftarrow \rho\left(p^{(d+1)} || r_d\right)$, append $\left(r_{d+1}, r_{d+1}^2, \cdots, r_{d+1}^{2^{\mu - d \cdot s - 1}}\right)$ to $\vec{r}$.
  5) Define vector $\vec{\pi}$ of length n, where $\vec{\pi}[i] = \left(\left\{\mathsf{MT.Open}(i \bmod (n/2^{k \cdot s}), \vec{mt}^{(k)})\right\}_{k \in [1,d]}, p^{(d+1)}(X)\right)$.
  6) Output $(\vec{r}, \vec{\pi})$.
- $\mathsf{Verify}(i, \vec{c}[i], \vec{\pi}[i]) \to (\vec{r}, y_r)/\perp$: Given the index i, its claimed RS code element $\vec{c}[i]$ at the i-th position, and consolidation proof $\vec{\pi}[i]$, outputs randomized point $\vec{r}$ and new evaluation claim y_r to accept, or $\perp$ to reject.
  1) Parse $\left(\left\{\mathsf{path}_k, p^{(k)}(X)\right\}_{k \in [1,d]}, p^{(d+1)}(X)\right) \leftarrow \vec{\pi}[i]$ into Merkle tree paths and leaves, and final polynomial.
  2) Let $\vec{r}$ be empty vector. Let ω_0 denote the n-th root of unity and $\omega_k = \omega_{k-1}^{2^s}$. For simplicity, define $p^{(0)}(r_0) := \vec{c}[i]$.
  3) For $k \in [1, d]$:
     a) If $p^{(k)}(\omega_{k-1}^i) \neq p^{(k-1)}(r_{k-1})$, outputs $\perp$.
     b) Derive $r_k \leftarrow \rho\left(\mathsf{MT.Recover}(\mathsf{path}_k, p^{(k)}, i \bmod (n/2^{k \cdot s})) || r_{k-1}\right)$. Append $\left(r_k^{2^0}, r_k^{2^1}, \cdots, r_k^{2^{s-1}}\right)$ to $\vec{r}$.
  4) If $p^{(d+1)}(\omega_d^i) \neq p^{(d)}(r_d)$, outputs $\perp$.
  5) Derive $r_{d+1} \leftarrow \rho\left(p(X) || r_d\right)$. Append $\left(r_{d+1}, r_{d+1}^2, \cdots, r_{d+1}^{2^{\mu - d \cdot s - 1}}\right)$ to $\vec{r}$.
  6) Output $\vec{r}$, $y_r = p(r_{d+1})$.

It remains to link the Merkle commitment of $\left\{\tilde{f}_{\vec{b}}\right\}$ with the group commitment to $\tilde{f}'$. We do so by sampling a subset I of s random columns from the interleaved RS codeword and checking that the linear combination of the corresponding $\{\tilde{f}_{\vec{b}}\}$ coefficients (with weights $\prod_{i=1}^{\mu'} z_i^{\vec{b}[i]}$) is consistent with the evaluation of $\tilde{f}'$. Concretely, for every $i \in I$, prover:

1) opens the i-th column of committed $\tilde{f}$ via $\mathsf{MT.Open}$, where verifier can compute

$$y_i = \sum_{\vec{b} \in \{0,1\}^{\mu'}} f_{\vec{b}}(\omega^i)$$

2) invokes $\mathsf{PC.BatchEval}$ to prove

$$\tilde{f}'(\omega^i, \omega^{2i}, \cdots, \omega^{2^{\mu - \mu'} \cdot i}) = y_i$$

where ω is $(2^{\mu - \mu'}/\rho)$-th root of unity.

The complete LightLigero protocol is presented in Protocol 3. This can be made non-interactive with Fiat–Shamir transformation [47].

5.2. Analysis

Theorem 3 (Soundness)**.** *Let I be the set of sampled columns with $|I| = s$. Then* LightLigero *has soundness error at most $\rho^s + \mathsf{negl}(\lambda)$.*

Proof sketch. If the interleaved RS commitment is inconsistent with $\tilde{f}'$, then with probability at least $1 - \rho^s$ the sampled columns expose a disagreement in one of the checked slices; the residual term is bounded by the security λ of $\mathsf{PST.BatchEval}$. A full proof appears in Appendix D. For instance, with $\rho = 1/4$ and $s = 50$, we obtain ≈ 100-bit soundness from the code test alone.

Cost of LightLigero**.** We summarize asymptotic and concrete costs.

- **Prover time.** *Commit:* encoding $2^{\mu'}$ rows and building a Merkle tree over $(2^{\mu - \mu'}/\rho)$ columns costs $O(n \log n)\,\mathbb{F} + O(n)\,\mathbb{H}$. *Open:* committing and opening $\tilde{f}'$ with PC^G costs $O(\frac{n}{\log n})\,\mathbb{G}$. *Batch checks:* s evaluations of $\tilde{f}'$ add $O(\frac{sn}{\log n})$ field work, negligible compared to $O(n \log n)\,\mathbb{F}$

Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.

Protocol 2. UltraProofs: Interleaved RS Code Commitment Scheme

Setup: $\mathsf{Setup}(1^\lambda) \to \mathsf{pp}$: Given the security parameter and output $\mathsf{pp} := (\mathsf{PC.Setup}(1^\lambda), \Omega, s, n)$.

Commit: $\mathsf{Commit}(\mathsf{pp}, M) \to \mathsf{com}$:

1) Parses the input $M \in \mathbb{F}^{L\times k}$ as a bivariate polynomial $f(X,Y)$ such that $f(X,Y) = \sum_{i\in[L],j\in[k]} M[i,j]\cdot X^i Y^j$.
2) Let $\tilde{f}$ be the twin polynomial of $f(X,Y)$, compute $\mathsf{com} \leftarrow \mathsf{PC.Commit}(\mathsf{pp}, \tilde{f})$.

Prove: $\mathsf{Prove}(\mathsf{pp}, M) \to \vec{\pi}$. This algorithm directly generate proofs for all n RS code shares.

1) Encode $M \in \mathbb{F}^{L\times k}$ into interleaved RS codeword $\vec{c} = (\vec{c}^{(0)}, \cdots, \vec{c}^{(n-1)}) \in \mathbb{F}^{L\times n}$.
2) Construct vector $\vec{v}$ of length n, such that its i-th element is a univariate polynomial $c_i(X) = \sum_{j\in[L]} \vec{c}^{(i)}[j]\cdot X^j$ whose coefficients are $\vec{c}^{(i)}$.
3) Construct Merkle tree $mt \leftarrow \mathsf{MT.Construct}(\vec{v})$. Define the i-th path of mt as path_i.
4) Derive $\beta \leftarrow \rho(\mathsf{com}||mt.\mathsf{Root})$. Consolidate all evaluations: $(\vec{r}, \vec{\pi}_{\mathsf{EC}}) \leftarrow \mathsf{NIEC.Prove}(\vec{c}, \Omega)$ where $\vec{c}[j] = c_j(\beta)$.
5) Run evaluation on the consolidated point: $(y_r, \pi_{\mathsf{pc}}) \leftarrow \mathsf{PC.Eval}(\mathsf{pp}, \vec{r}, \tilde{f})$.
6) Outputs $\vec{\pi}$ of length n such that $\vec{\pi}[i] := (\mathsf{path}_i, \vec{\pi}_{\mathsf{EC}}[i], \pi_{\mathsf{pc}})$.

Verify: $\mathsf{Verify}(\mathsf{pp}, \mathsf{com}, i, c_i(X), \vec{\pi}[i]) \to 0/1$

1) Parse $(\mathsf{path}, \pi_{\mathsf{EC}}, \pi_{\mathsf{pc}}) \leftarrow \pi$. Derive $\beta \leftarrow \rho(\mathsf{MT.Recover}(\mathsf{path}, c_i(X), i)\|\mathsf{com})$.
2) Invoke $(\vec{r}, y_r) \leftarrow \mathsf{NIEC.Verify}(j, c_j(\beta), \vec{r}, \pi_{\mathsf{EC}})$ to verify the evaluation consolation proof. If it outputs $\perp$, return 0.
3) Outputs $\mathsf{PC.Verify}(\mathsf{com}, (\vec{\beta}||\vec{r}), y_r, \pi_{\mathsf{pc}})$, where $\vec{\beta} = (\beta^{2^0}, \beta^{2^1}, \cdots, \beta^{L/2})$.

in typical parameter regimes. Thus the total prover cost is

$$O(n\log n)\cdot\mathbb{F} + O(n)\cdot\mathbb{H} + O(\frac{n}{\log n})\cdot\mathbb{G}$$

Concretely, while PC^G (e.g. PST) alone is roughly $3\times$ slower than Deepfold on $n = 2^{20}$, the PST component touches only $n/\log n$ coefficients; for $\log n = 16$ it increases end-to-end prover time by about 20%.

- **Proof size.** The proof consists of: a PST commitment and opening for $\tilde{f}'$ ($O(\log n)$ scalars), a batched evaluation proof ($O(\log n)$), and $s = \Theta(\lambda)$ Merkle openings of depth $O(\log n)$. Hence the total size is $O(\lambda \log n)$. For $n = 2^{20}$, choosing $s = 50$ and Merkle leaf arity 2 (depth 16), we send about $50 \times 16 = 800$ field elements (opened leaves) and 800 hashes. At 32 bytes per scalar/hash, this is $\approx 32 \times 1600 \approx 51.2$ KB, over $4\times$ smaller than Deepfold under comparable settings.

6. Applications

We highlight three representative applications of *UltraProofs*: verifiable secret sharing (VSS), blockchain data availability (DA), and multi-verifier zkSNARKs. All three rely on a common capability—dispersing encoded data among many participants while providing succinct, locally verifiable proofs of correctness for each share.

6.1. Verifiable Secret Sharing

Secret sharing enables a dealer to distribute a secret among n participants such that any subset of at least k participants can reconstruct the secret, while smaller subsets learn nothing. Typically, n is set to $2k$ to achieve optimal fault tolerance in synchronous Byzantine settings. A *verifiable secret sharing* (VSS) protocol additionally guarantees *verifiability*: even if the dealer is malicious, honest participants can locally verify that their received shares are consistent with a well-defined secret.

Modern VSS protocols commonly rely on Reed–Solomon (RS) code commitments. The dealer encodes a secret vector $\vec{m}$ into a codeword $\vec{c}$ using $\mathsf{RS}[k,n]$, broadcasts a commitment to $\vec{m}$, and privately sends each participant their corresponding share $\vec{c}[i]$ together with an opening proof. Each participant locally verifies that their received share matches the commitment, ensuring correctness and non-equivocation without requiring trust in the dealer.

UltraProofs integrates naturally as a commitment primitive for VSS, offering both succinct proofs and linear-time encoding. To additionally guarantee *privacy* against coalitions of fewer than k participants, we extend the committed object to include a random masking polynomial.

Concretely, the dealer constructs a single multilinear polynomial $\tilde{h}$ that jointly encodes two k-length components:

$$\tilde{h}(Y, \vec{X}) = \tilde{f}(\vec{X}) + Y\cdot\tilde{g}(\vec{X}),$$

where $\tilde{f}$ represents the secret polynomial and $\tilde{g}$ is a random masking polynomial sampled uniformly from the same function space. The dealer commits once to $\tilde{h}$ using multilinear PCS, obtaining the commitment com_h.

To distribute shares, the dealer derives a random scalar $\beta \in \mathbb{F}$ (via Fiat–Shamir or a public coin) and, for each participant i, sends:

1) the actual secret share $\tilde{f}(\omega^i)$,
2) the masked evaluation $\tilde{g}(\omega^i)$,
3) an opening proof that $\tilde{h}(\omega^i, \beta)$ is consistent with com_h.

Opening proofs in the third part only transmit evaluations of $\tilde{f}(\omega^i) + \beta\tilde{g}(\omega^i)$. Since $\tilde{g}$ is uniformly random, the linear combination $\tilde{f} + \beta\tilde{g}$ is statistically independent of $\tilde{f}$. Even if adversaries obtain the entire polynomial $\tilde{f} + \beta\tilde{g}$, perfect secrecy of $\tilde{f}$ is still preserved.

Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.

Protocol 3. Multilinear polynomial commitment LightLigero

- $\mathsf{Commit}(\tilde{f}) \to \mathcal{C}$: Given the μ-variate multilinear polynomial $\tilde{f}$, prover generates commitment of $\tilde{f}$.
  1) Assume the coefficient of $\tilde{f}$ can be arranged into matrix $C \in \mathbb{F}^{2^{\mu'} \times 2^{\mu-\mu'}}$ such that $\mu' = \log \mu$.
  2) Define $m = 2^{\mu-\mu'}/\rho$, where ρ is the coderate of RS code. Compute matrix $C' \in \mathbb{F}^{2^{\mu'} \times m}$ such that $\forall i \in [2^{\mu'}]$, $C'[i,:] = \mathsf{RS}(C[i,:], \rho)$.
  3) Construct Merkle tree mt with m leaves, the j-th leaf is μ'-variate multilinear polynomial $\tilde{p}_i$ whose coefficient is $C'[:,j]$. Output $\mathcal{C} = mt.\mathsf{Root}$ as commitment of $\tilde{f}$.
- $\left\langle \mathsf{Eval}(\tilde{f}), \mathsf{Verify}(\mathcal{C}) \right\rangle (\vec{z} = (z_1, \cdots, z_\mu), y) \to 0/1$: $\mathcal{P}$ takes the polynomial $\tilde{f}$ and the random evaluation point $\vec{z}$ as input, $\mathcal{V}$ takes commitment $\mathcal{C}$ as input, outputs 0/1 indicating whether accept the proof.
  1) Given another group-based multilinear polynomial PC^G with slower prover time and smaller proof size, $\mathcal{P}$ invokes $\mathcal{C}' \leftarrow \mathsf{PC}^G.\mathsf{Commit}$ to commit $\mu - \mu'$ variate multilinear polynomial $\tilde{f}' = \tilde{f}(z_1, \cdots, z_{\mu'}, X_{\mu'+1}, \cdots, X_\mu)$. $\mathcal{P}$ sends $\mathcal{C}'$ to $\mathcal{V}$.
  2) $\mathcal{V}$ samples a random subset $I \subseteq [m]$ such that $|I| = s$, and sends I to $\mathcal{P}$. $\mathcal{V}$ samples uniform random $\alpha \in \mathbb{F}$ and sends α to $\mathcal{P}$.
  3) $\mathcal{P}$ invokes $\{(\tilde{p}_i, \mathsf{path}_i)\}_{i \in I} \leftarrow \mathsf{MT.Open}(I, mt)$ and sends them to $\mathcal{V}$. $\mathcal{V}$ checks the correctness of these Merkle paths. If one of these paths is incorrect, $\mathcal{V}$ outputs 0 and the protocol terminates. For $\forall j \in [s]$, $\mathcal{V}$ computes $y_j = \tilde{p}_{I[j]}(z_1, \cdots, z_{\mu'})$.
  4) $\mathcal{P}$ and $\mathcal{V}$ perform batched open of $\tilde{f}'(z_{\mu'+1}, \cdots, z_\mu)$ and $f'(\omega^i)$ for all $i \in I$. Specifically, $\mathcal{P}$ and $\mathcal{V}$ perform sumcheck for
  $$\sum_{\vec{b} \in \{0,1\}^{\mu-\mu'}} \tilde{f}'(\vec{b}) \cdot \left(\tilde{eq}_{\vec{z}'}(\vec{b}) + \alpha \cdot \sum_{j \in [s]} \alpha^j \tilde{eq}_j(\vec{b}) \right) = y + \alpha \cdot \sum_{j \in [s]} \alpha^j \cdot y_i$$
  where $\vec{z}' := (z_{\mu'+1}, \cdots, z_\mu)$, $\tilde{eq}_j(\cdot) := \tilde{eq}_{\vec{\omega}_{I[j]}}(\cdot)$, $\vec{\omega}_i := (\omega^i, \omega^{2i}, \omega^{4i}, \cdots, \omega^{2^{\mu-\mu'} \cdot i})$. This sumcheck will reduce the batched evaluations to claimed evaluation for $\tilde{f}'(r_1, \cdots, r_{\mu-\mu'})$.
  5) $\mathcal{P}$ invokes $\mathsf{PC}^G.\mathsf{Eval}$ to prove the evaluation of $\tilde{f}'(r_1, \cdots, r_{\mu-\mu'})$ and sends the proof to $\mathcal{V}$.
  6) If the proof can't pass, $\mathcal{V}$ outputs 0; otherwise outputs 1.

Compared to HydraProofs [24], UltraProofs eliminates redundant commitments and avoids committing to the entire RS codeword. HydraProofs must commit separately to both $\vec{m}$ and $\vec{c}$, effectively doubling the commitment overhead. Since RS encoding and evaluation consolidation is far cheaper than polynomial commitment, the overall prover runtime of UltraProofs is empirically about $3\times$ faster than HydraProofs under standard VSS parameters, as demonstrated in our experimental evaluation. This efficiency makes UltraProofs a practical and scalable commitment layer for verifiable secret sharing in distributed and blockchain-based environments.

6.2. Data Availability

Data availability (DA) is a fundamental security requirement in blockchain systems. Before finalizing a block, validators must be assured that the block's contents are publicly retrievable even if the proposer is adversarial. A naïve approach is to replicate the entire block across all validators, which guarantees availability but incurs prohibitive bandwidth and storage costs as block sizes and transaction throughput scale, ultimately limiting network decentralization.

RS-coded data availability. Modern DA protocols mitigate this overhead by encoding each block using a RS code and distributing encoded fragments to storage nodes. Each node stores only a small subset of the codeword symbols, while the erasure-resilient property of RS codes ensures that the full block can be reconstructed as long as a sufficient fraction of fragments remain available. Concretely, if the original block data are represented as a matrix $M \in \mathbb{F}^{L \times k}$, existing systems such as NNT [19], ESPRESSO [48], and FRIDA [16] employ *interleaved* RS codes to encode M, assigning each storage node several columns of the resulting codeword. To guarantee correctness, these systems rely on RS code commitments, enabling each node to verify that its received data fragment corresponds to a valid RS encoding of the committed block.

Limitations of prior schemes. While FRIDA [16] achieves fast prover time via FRI-based Interactive Oracle Proofs of Proximity (IOPPs), its per-node proof size remains large—on the order of hundreds of kilobytes. This overhead is negligible when the number of nodes is small (i.e., when each node stores larger data fragments), but becomes prohibitive in large-scale settings with thousands of storage nodes, where the proof size can exceed the data fragment itself. For example, when dispersing a $64\,\mathrm{MB}$ block across 2048 nodes[4], the per-node communica-

4. Setup: $k = 2^{16}$, $L = 2^5$, $\rho = 0.25$, $\mathbb{F}$ is the scalar field of BN254. The 64 MB data are expanded into a 256 MB encoded block.

Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.

tion cost in FRIDA—including commitments and opening proofs—totals approximately 664 KB, while each node's data fragment is only 128 KB. This 5.7× overhead severely limits scalability and would be even more pronounced in networks such as Ethereum, which currently has over 6000 active validators[5].

UltraProofs for scalable DA. UltraProofs addresses this inefficiency by replacing FRIDA's FRI-based proof system with a PCS-agnostic RS code commitment, drastically reducing per-node communication. When instantiated with LightLigero as the underlying PCS, UltraProofs achieves several-fold reductions in proof size with only a modest increase in prover computation—yielding a more practical trade-off for decentralized storage environments. Under the same parameters as the above example, UltraProofs with LightLigero reduces the per-node communication to roughly 186 KB, cutting the total bandwidth by more than 3.5× while preserving transparency and compatibility with standard cryptographic assumptions.

6.3. Multi-Verifier zkSNARKs

The evaluation consolidation technique of UltraProofs can also be applied to multi-party verification, where multiple verifiers each hold distinct pieces of input data contributing to a common computation.

Consider a setting with a single prover $\mathcal{P}$ holding the vector $\vec{x} = (x_1, \ldots, x_n)$ and n verifiers $\{\mathcal{V}_1, \ldots, \mathcal{V}_n\}$, where each $\mathcal{V}_j$ possesses its local input x_j. The prover wishes to convince all verifiers that there exists a witness $\vec{w}$ satisfying a relation

$$\mathcal{C}(\vec{x}, \vec{w}) = 1,$$

while also ensuring that each verifier's local input x_j is correctly embedded in the committed vector $\vec{x}$. To do so, the prover must generate:

1) n proofs showing that each verifier's input x_j corresponds to the j-th segment of the committed vector com_x, and
2) a zkSNARK proof attesting that $\mathcal{C}(\vec{x}, \vec{w}) = 1$ and that $\vec{x}$ is consistent with com_x.

Let $\tilde{f}$ denote the multilinear extension of $\vec{x}$ over μ variables. Our evaluation-consolidation protocol can reduce all 2^μ point evaluations of $\tilde{f}$ at $\{0,1\}^\mu$—corresponding to the local inputs held by 2^μ verifiers—into a single evaluation $\tilde{f}(\vec{r})$ at a common random point $\vec{r} \in \mathbb{F}^\mu$, along with 2^μ succinct reduction proofs. In the i-th round, the prover sends the message $\tilde{f}(\vec{r}_{[:i-1]}, X, \vec{b}_{[i+1:]})$ to all verifiers whose indices share the suffix $\vec{b}_{[i+1:]}$. This structure enables all verifiers to share the same verifier randomness while maintaining soundness. Since many modern zkSNARK constructions [50] use multilinear PCS commitments to represent circuit wire values, UltraProofs can be seamlessly integrated into existing zkSNARK frameworks with negligible additional cost.

5. As of April 2025, 6680 validators are active on the Ethereum mainnet [49].

Compared to HydraProofs [45], UltraProofs eliminates the reliance on homomorphic polynomial commitments, significantly improving efficiency and flexibility in instantiation. This allows UltraProofs to operate over smaller characteristic fields [44], [51] or to adopt post-quantum–secure PCS [52] backends, depending on deployment needs. This versatility makes UltraProofs a practical drop-in component for applications such as secure multiparty computation [53], [54] and robust aggregation [55], [56] in decentralized systems.

7. Evaluation

We implemented and experimentally evaluated two variants of UltraProofs and LightLigero, in the context of verifiable secret sharing (VSS) and blockchain data availability (DA).

Software. Our implementation is written in Rust, comprising approximately 1.5K lines of code. The source code is publicly available at https://github.com/guo-yanpei/ultraproofs. We rely on the Arkworks library [31] for finite field and elliptic-curve operations, instantiated over the BN254 curve, which provides roughly 100 bits of security. For the VSS and DA experiments, we benchmark only the corresponding RS-code commitment in order to match the baseline implementations.

Hardware. All experiments were performed locally on a MacBook M1 laptop equipped with 16GB of RAM, using a single CPU thread.

7.1. Verifiable Secret Sharing

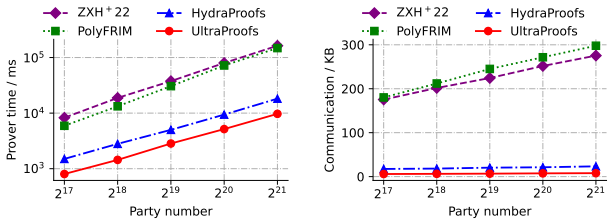


Figure 1: Single-threaded prover time (left) and per-share communication cost (right) of transparent VSS schemes.

Verifiable secret sharing is as a direct application of single-layer RS code commitments. We benchmark UltraProofs (instantiated with Dory PCS) against three representative VSS schemes: ZXH+22 [17], PolyFRIM [18], and HydraProofs. Dory is chosen as our multilinear PCS due to its transparent setup and fast $O(n) \cdot \mathbb{F}$ evaluation time. The number of receivers n ranges from 2^{17} to 2^{21}, with the reconstruction threshold fixed at $k = n/2$, corresponding to optimal Byzantine fault tolerance.

Figure 1 summarizes the results. UltraProofs achieves the fastest dispersal time (including RS encoding and proof generation) and the smallest communication cost among

Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.

all schemes. For $n = 2^{21}$, UltraProofs completes all proof generation in only 8.2s, while HydraProofs requires 19.6s—approximately $2.4\times$ slower. This performance gap arises from HydraProofs' redundant commitments to both $\vec{m}$ and $\vec{c}$. In terms of communication, UltraProofs achieves 6.5 KB per verifier, compared to HydraProofs' 24 KB, representing a $3.7\times$ reduction. The improvement stems from the $O(\log \log n)$ asymptotic advantage of our consolidation procedure.

Against ZXH+22 (using Virgo [57]) and PolyFRIM, UltraProofs achieves at least $6\times$ faster prover time and more than $30\times$ smaller proof size, owing to its PCS agnosticism, which enables the combination of evaluation consolidation with a communication-efficient Dory PCS.

All schemes complete verification within 10 ms; since this cost is negligible, we omit it from the plots.

7.2. Data Availability

We next evaluate the combination of UltraProofs and LightLigero in a data-availability setting, which corresponds to the interleaved RS code commitment scenario.

Parameter choice. Industrial DA systems such as Celestia [58] and EigenDA [59] currently support block sizes of around 15MB. To reflect near-future scalability targets, we evaluate block sizes ranging from 32MB to 128MB. Given that a scalar field element of BN254 can approximately encode 32 bytes of information, this block size range corresponds to polynomials of length (i.e. $k \times L$ in the context of interleaved RS code) 2^{20} to 2^{22}. To emulate realistic decentralization[6], we vary the number of DA nodes N from 1024 to 4096, which corresponds to the number of interleaved RS codeword columns n ranging from 2^{10} to 2^{12}. The code rate is fixed at $\rho = 1/4$, consistent with common DA security settings [16], [48]. We measure the cost of RS code commitments under these DA configurations.

Table 3 reports concrete performance of FRIDA and UltraProofs. When paired with our custom PCS LightLigero, UltraProofs achieves a $2\sim5\times$ reduction in per-node communication compared to FRIDA, with only a $\leq 25\%$ increase in prover time. For UltraProofs+Dory, communication drops by roughly $6\times$ relative to FRIDA, albeit at a $3\times$ slower prover. The two instantiations thus reflect a tunable trade-off between proof size and prover complexity, governed by the choice of underlying PCS. In all settings, verifier time remains under 50ms.

The trade-off of slightly increasing prover time for reduced communication is advantageous in DA, not only from a numerical perspective, but also because per-node communication is inherently more critical than prover time. Specifically, per-node communication imposes a bandwidth constraint on each DA client, whereas the prover cost is borne solely by the disperser. Since the primary goal of DA is to enable participation in blockchain consensus by more resource-constrained clients, lowering bandwidth requirements is more crucial than minimizing prover time. Moreover, the higher prover cost can be mitigated through parallelization.

6. Ethereum today has 6800 detected participating nodes as of October, 2025 [49]

7.3. Multilinear Polynomial Commitment

Finally, we implement our multilinear PCS, LightLigero, by integrating it with PST [32], and compare it with Deepfold [28] and WHIR [29]. All three are RS-code-based constructions with similar asymptotic prover complexity. We ranges the polynomial size from 2^{19} to 2^{24}, covering block size currently and in the near future. The code rate is fixed at $1/4$.

As shown in Figure 2, the prover time of LightLigero is about 30% slower than Deepfold due to the inclusion of additional multi-scalar multiplications, yet it remains $1.8\times$ faster than WHIR, whose evaluation step involves heavy IFFT/FFT operations. In terms of proof size, LightLigero achieves $4\times$ compression relative to Deepfold and $2\times$ relative to WHIR, owing to its asymptotically smaller $O(\lambda \log n)$ bound. For a polynomial of size 2^{21} (64 MB of data), Deepfold produces 225 KB proofs, WHIR 104 KB, and LightLigero only 54 KB. The verifier time of LightLigero is roughly $3\times$ and $11\times$ longer than that of Deepfold and WHIR, respectively. However, since all of them are below 10 ms, this will not become a system bottleneck in UltraProofs's target applications.

8. Conclusion

In this work, we presented *UltraProofs*, a framework for scalable and PCS-agnostic Reed–Solomon code commitments. By introducing a non-interactive *evaluation consolidation* protocol, UltraProofs avoids redundant codeword commitments and removes the reliance on homomorphic PCS backends, achieving asymptotically optimal $O(n)$ proof generation while remaining compatible with any multilinear PCS. To improve concrete efficiency, we further developed *LightLigero*, an RS code-based multilinear PCS that achieves $O(\lambda \log n)$ proof size with $O(n \log n)$ prover time, reducing communication overhead relative to prior hash-based designs.

We evaluated UltraProofs in the contexts of verifiable secret sharing and blockchain data availability. In VSS, UltraProofs achieves a $2.4\times$ reduction in prover time and a $4\times$ reduction in proof size compared to HydraProofs (S&P'25). In DA, UltraProofs combined with LightLigero reduces per-node communication by $2\sim5\times$ compared to FRIDA (Crypto'24).

These results indicate that UltraProofs provides a practical and modular foundation for large-scale verifiable data dispersal, while maintaining compatibility with a broad class of polynomial commitment schemes.

Acknowledgement

The authors acknowledge funding support from CRPO (WBS A-8004052-01-00) and MOE grant MOE-T2EP20125-0015.

Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.

TABLE 3: Performance of UltraProofs in data availability applications.

	$N = 1024$			$N = 2048$			$N = 4096$		
Block size $\|M\|$	32MB	64MB	128MB	32MB	64MB	128MB	32MB	64MB	128MB
Prover (s)									
FRIDA [16]	2.39	5.04	12.06	2.34	4.92	11.04	2.37	5.08	12.47
UltraProofs + Dory	8.80	16.87	34.50	8.98	17.16	33.15	8.94	17.24	34.09
UltraProofs + LightLigero	2.88	5.81	13.03	2.91	5.95	11.81	2.86	6.03	13.17
Per-node Comm. (KB)									
FRIDA [16]	598.74	792.02	1016.70	534.74	624.02	810.70	552.74	640.02	832.70
UltraProofs + Dory	134.98	261.08	517.17	67.98	132.08	260.17	36.63	68.73	132.82
UltraProofs + LightLigero	194.63	324.77	575.91	120.63	186.77	316.91	69.28	123.42	189.56
Verifier (ms)									
FRIDA [16]	21	33	48	19	27	35	20	27	36
UltraProofs + Dory	9	8	10	7	8	10	7	9	8
UltraProofs + LightLigero	13	21	36	14	22	36	12	21	34

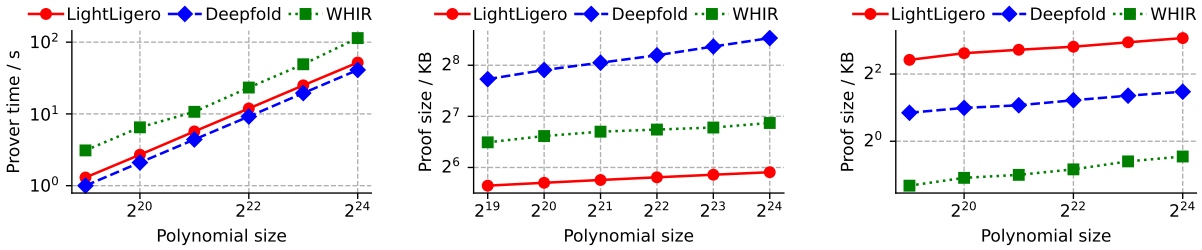


Figure 2: Single-threaded prover time, proof size and verifier time of PCS schemes.

Ethics Considerations

None.

LLM usage considerations

LLMs were used for editorial purposes in this manuscript, and all outputs were inspected by the authors to ensure accuracy and originality.

References

[1] I. S. Reed, G. Solomon, and K. H. March, "Polynomial codes over certain finite fields," *Journal of The Society for Industrial and Applied Mathematics*, vol. 8, pp. 300–304, 1960.

[2] M. O. Rabin, *The information dispersal algorithm and its applications*. Berlin, Heidelberg: Springer-Verlag, 1990, p. 406–419.

[3] J. A. Garay, R. Gennaro, C. Jutla, and T. Rabin, "Secure distributed storage and retrieval," in *Distributed Algorithms*, M. Mavronicolas and P. Tsigas, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1997, pp. 275–289.

[4] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[5] P. Feldman, "A practical scheme for non-interactive verifiable secret sharing," in *28th Annual Symposium on Foundations of Computer Science (sfcs 1987)*. IEEE, 1987, pp. 427–438.

[6] T. Yurek, L. Luo, J. Fairoze, A. Kate, and A. Miller, "hbacss: How to robustly share many secrets," in *Proceedings of the Network and Distributed System Security Symposium (NDSS) 2022*, 2022.

[7] C. Cachin and S. Tessaro, "Asynchronous verifiable information dispersal," in *24th IEEE Symposium on Reliable Distributed Systems (SRDS'05)*. IEEE, 2005, pp. 191–201.

[8] N. Alhaddad, S. Das, S. Duan, L. Ren, M. Varia, Z. Xiang, and H. Zhang, "Asynchronous verifiable information dispersal with near-optimal communication," Cryptology ePrint Archive, Paper 2022/775, 2022. [Online]. Available: https://eprint.iacr.org/2022/775

[9] N. Alhaddad, S. Duan, M. Varia, and H. Zhang, "Succinct erasure coding proof systems," 2021, https://eprint.iacr.org/2021/1500.

[10] M. Al-Bassam, A. Sonnino, V. Buterin, and I. Khoffi, "Fraud and data availability proofs: Detecting invalid blocks in light clients," in *Financial Cryptography and Data Security: 25th International Conference, FC 2021, Virtual Event, March 1–5, 2021, Revised Selected Papers, Part II 25*. Berlin, Heidelberg: Springer-Verlag, 2021, pp. 279–298.

[11] L. Yang, S. J. Park, M. Alizadeh, S. Kannan, and D. Tse, "DispersedLedger: High-Throughput byzantine consensus on variable bandwidth networks," in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. Renton, WA: USENIX Association, 2022, pp. 493–512. [Online]. Available: https://www.usenix.org/conference/nsdi22/presentation/yang

[12] V. Buterin, "2d data availability with kate commitments," https://ethresear.ch/t/2d-data-availability-with-kate-commitments/8081, October 2020, ethereum Research Forum Post.

[13] J. Hendricks, G. R. Ganger, and M. K. Reiter, "Verifying distributed erasure-coded data," in *Proceedings of the Twenty-Sixth Annual ACM Symposium on Principles of Distributed Computing*. New York, NY, USA: Association for Computing Machinery, 2007. [Online]. Available: https://doi.org/10.1145/1281100.1281122

[14] A. Tomescu, "How to compute all pointproofs," Cryptology ePrint Archive, Paper 2020/1516, 2020. [Online]. Available: https://eprint.iacr.org/2020/1516

[15] M. Hall-Andersen, M. Simkin, and B. Wagner, "Foundations of data availability sampling," Cryptology ePrint Archive, Paper 2023/1079, 2023. [Online]. Available: https://eprint.iacr.org/2023/1079

[16] ——, "Frida: Data availability sampling from fri," in *Annual International Cryptology Conference*. Berlin, Heidelberg: Springer-Verlag, 2024, pp. 289–324.

Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.

[17] J. Zhang, T. Xie, T. Hoang, E. Shi, and Y. Zhang, "Polynomial commitment with a One-to-Many prover and applications," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, 2022, pp. 2965–2982. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/zhang-jiaheng

[18] Z. Zhang, W. Li, Y. Guo, K. Shi, S. S. M. Chow, X. Liu, and J. Dong, "Fast RS-IOP multivariate polynomial commitments and verifiable secret sharing," in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, 2024, pp. 3187–3204. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/zhang-zongyang

[19] K. Nazirkhanova, J. Neu, and D. Tse, "Information dispersal with provable retrievability for rollups," in *Proceedings of the 4th ACM Conference on Advances in Financial Technologies*, ser. AFT '22. New York, NY, USA: Association for Computing Machinery, 2023, p. 180–197. [Online]. Available: https://doi.org/10.1145/3558535.3559778

[20] A. Evans, N. Mohnblatt, and G. Angeris, "ZODA: Zero-overhead data availability," Cryptology ePrint Archive, Paper 2025/034, 2025. [Online]. Available: https://eprint.iacr.org/2025/034

[21] A. Kate, G. M. Zaverucha, and I. Goldberg, "Constant-size commitments to polynomials and their applications," in *Advances in Cryptology - ASIACRYPT 2010*, M. Abe, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 177–194.

[22] A. Tomescu, R. Chen, Y. Zheng, I. Abraham, B. Pinkas, G. G. Gueta, and S. Devadas, "Towards scalable threshold cryptosystems," in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 877–893.

[23] D. Feist and D. Khovratovich, "Fast amortized KZG proofs," Cryptology ePrint Archive, Paper 2023/033, 2023. [Online]. Available: https://eprint.iacr.org/2023/033

[24] C. Pappas, D. Papadopoulos, and C. Papamanthou, "Hydraproofs: Optimally computing all proofs in a vector commitment (with applications to efficient zksnarks over data from multiple users)," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 3421–3439.

[25] S. Goldwasser, Y. T. Kalai, and G. N. Rothblum, "Delegating computation: interactive proofs for muggles," *Journal of the ACM (JACM)*, vol. 62, no. 4, pp. 1–64, 2015.

[26] T. Liu, X. Xie, and Y. Zhang, "Zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 2968–2985.

[27] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Fast reed-solomon interactive oracle proofs of proximity," in *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, vol. 107. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018, pp. 14:1–14:17. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICALP.2018.14

[28] Y. Guo, X. Liu, K. Huang, W. Qu, T. Tao, and J. Zhang, "DeepFold: Efficient multilinear polynomial commitment from reed-solomon code and its application to zero-knowledge proofs," Cryptology ePrint Archive, Paper 2024/1595, 2024. [Online]. Available: https://eprint.iacr.org/2024/1595

[29] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogev, "WHIR: Reed–solomon proximity testing with super-fast verification," Cryptology ePrint Archive, Paper 2024/1586, 2024. [Online]. Available: https://eprint.iacr.org/2024/1586

[30] U. Haböck, "A summary on the FRI low degree test," Cryptology ePrint Archive, Paper 2022/1216, 2022. [Online]. Available: https://eprint.iacr.org/2022/1216

[31] arkworks contributors, "`arkworks` zksnark ecosystem," 2022. [Online]. Available: https://arkworks.rs

[32] C. Papamanthou, E. Shi, and R. Tamassia, "Signatures of correct computation," in *Theory of Cryptography*, A. Sahai, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 222–242.

[33] J. Lee, "Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments," in *Theory of Cryptography Conference*. Berlin, Heidelberg: Springer-Verlag, 2021, pp. 1–34.

[34] R. S. Wahby, I. Tzialla, A. Shelat, J. Thaler, and M. Walfish, "Doubly-efficient zksnarks without trusted setup," in *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2018, pp. 926–943.

[35] R. Bhadauria, Z. Fang, C. Hazay, M. Venkitasubramaniam, T. Xie, and Y. Zhang, "Ligero++: A new optimized sublinear iop," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 2025–2038.

[36] B. Chen, B. Bünz, D. Boneh, and Z. Zhang, "Hyperplonk: Plonk with linear-time prover and high-degree custom gates," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2023, pp. 499–530.

[37] K. D. Bowers, A. Juels, and A. Oprea, "Proofs of retrievability: theory and implementation," in *Proceedings of the 2009 ACM Workshop on Cloud Computing Security*, ser. CCSW '09. New York, NY, USA: Association for Computing Machinery, 2009, p. 43–54. [Online]. Available: https://doi.org/10.1145/1655008.1655015

[38] T. Xie, Y. Zhang, and D. Song, "Orion: Zero knowledge proof with linear prover time," in *Annual International Cryptology Conference*. Springer, 2022, pp. 299–328.

[39] W. Qu, Y. Guo, and J. Zhang, "Lightning, field-agnostic super-efficient polynomial commitment scheme," *Cryptology ePrint Archive*, 2026.

[40] S. Srinivasan, A. Chepurnoy, C. Papamanthou, A. Tomescu, and Y. Zhang, "Hyperproofs: Aggregating and maintaining proofs in vector commitments," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3001–3018.

[41] I. S. Reed and G. Solomon, "Polynomial codes over certain finite fields," *Journal of the society for industrial and applied mathematics*, vol. 8, no. 2, pp. 300–304, 1960.

[42] C. Lund, L. Fortnow, H. Karloff, and N. Nisan, "Algebraic methods for interactive proof systems," *J. ACM*, vol. 39, no. 4, p. 859–868, Oct. 1992. [Online]. Available: https://doi.org/10.1145/146585.146605

[43] D. Feist and D. Khovratovich, "Fast amortized KZG proofs," Cryptology ePrint Archive, Paper 2023/033, 2023. [Online]. Available: https://eprint.iacr.org/2023/033

[44] H. Zeilberger, B. Chen, and B. Fisch, "Basefold: efficient field-agnostic polynomial commitment schemes from foldable codes," in *Annual International Cryptology Conference*. Berlin, Heidelberg: Springer-Verlag, 2024, pp. 138–169.

[45] A. Kothapalli and S. Setty, "Hypernova: Recursive arguments for customizable constraint systems," in *Annual International Cryptology Conference*. Berlin, Heidelberg: Springer-Verlag, 2024, pp. 345–379.

[46] W. Li, Z. Zhang, S. S. Chow, Y. Guo, B. Gao, X. Song, Y. Deng, and J. Liu, "Shred-to-shine metamorphosis in polynomial commitment evolution," *Cryptology ePrint Archive*, 2025.

[47] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Conference on the theory and application of cryptographic techniques*. Springer, 1986, pp. 186–194.

[48] J. Bearer, B. Bünz, P. Camacho, B. Chen, E. Davidson, B. Fisch, B. Fish, G. Gutoski, F. Krell, C. Lin, D. Malkhi, K. Nayak, K. Shen, A. Xiong, N. Yospe, and S. Long, "The espresso sequencing network: HotShot consensus, tiramisu data-availability, and builder-exchange," Cryptology ePrint Archive, Paper 2024/1189, 2024. [Online]. Available: https://eprint.iacr.org/2024/1189

[49] Ethereum, "Ethereum node tracker," https://etherscan.io/nodetracker.

Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.

[50] W. Qu, Y. Guo, Y. Ying, and J. Zhang, "Verfcnn, optimal complexity zksnark for convolutional neural networks," *Cryptology ePrint Archive*, 2025.

[51] A. Golovnev, J. Lee, S. Setty, J. Thaler, and R. S. Wahby, "Brakedown: Linear-time and field-agnostic snarks for r1cs," in *Annual International Cryptology Conference*. Springer, 2023, pp. 193–226.

[52] N. K. Nguyen and G. Seiler, "Greyhound: fast polynomial commitments from lattices," in *Annual International Cryptology Conference*. Springer, 2024, pp. 243–275.

[53] D. Chaum, C. Crépeau, and I. Damgard, "Multiparty unconditionally secure protocols," in *Proceedings of the twentieth annual ACM symposium on Theory of computing*, 1988, pp. 11–19.

[54] I. Abraham, G. Asharov, S. Patil, and A. Patra, "Detect, pack and batch: perfectly-secure mpc with linear communication and constant expected time," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2023, pp. 251–281.

[55] M. Rathee, C. Shen, S. Wagh, and R. A. Popa, "Elsa: Secure aggregation for federated learning with malicious actors," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 1961–1979.

[56] A. Roy Chowdhury, C. Guo, S. Jha, and L. van der Maaten, "Eiffel: Ensuring integrity for federated learning," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 2535–2549.

[57] J. Zhang, T. Xie, Y. Zhang, and D. Song, "Transparent polynomial delegation and its applications to zero knowledge proof," in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 859–876.

[58] Celestia, "Celestia da," https://blog.celestia.org/ginger/.

[59] EigenDA, "Eigen da," https://docs.eigenda.xyz/core-concepts/overview.

[60] T. Xie, J. Zhang, Y. Zhang, C. Papamanthou, and D. Song, "Libra: Succinct zero-knowledge proofs with optimal prover computation," in *Advances in Cryptology–CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III 39*. Springer, Cham, 2019, pp. 733–764.

[61] A. Kothapalli and B. Parno, "Algebraic reductions of knowledge," in *Annual International Cryptology Conference*. Springer, 2023, pp. 669–701.

Appendix A.
Security Properties of Commitment Schemes

A.1. Code Commitment

A security code commitment should satisfy the following completeness property: For every $\mathsf{gp} \in \mathsf{Setup}(1^\lambda)$, every $m \in \mathbb{F}^{L\times k}$, and every $i \in [n]$, we have

$$\Pr\left[\mathsf{Verify}(\mathsf{ck},\mathsf{com},i,\hat{m}_i,\tau)=1 \;\middle|\; \begin{array}{l}(\mathsf{com},St)\leftarrow\mathsf{Commit}(\mathsf{ck},m)\\ \hat{m}:=\mathsf{RS}(m)\\ \tau\leftarrow\mathsf{Prove}(\mathsf{ck},St,i)\end{array}\right]=1$$

Most importantly, erasure code commitments should be position-binding and code-binding, as defined next. Informally, position binding ensures that no adversary can open the codeword at one position to two different values. Code-binding states that whatever an adversary opens is consistent with the code, i.e., the objects to which one commits are really codewords.

Definition 3 (Position-Binding)**.** *We say that a RS code commitment* $\mathsf{CC}=(\mathsf{Setup},\mathsf{Commit},\mathsf{Prove},\mathsf{Verify})$ *is position-binding, if for every PPT algorithm* $\mathcal{A}$*, the following advantage is negligible:*

$$\Pr\left[\begin{array}{r|r}\hat{m}\neq\hat{m}'\wedge & \mathsf{gp}\leftarrow\mathsf{Setup}(1^\lambda)\\ \mathsf{Verify}(\mathsf{gp},\mathsf{com},i,\hat{m},\tau)=1\wedge & (\mathsf{com},i,\hat{m},\tau,\hat{m}',\tau')\\ \mathsf{Verify}(\mathsf{gp},\mathsf{com},i,\hat{m}',\tau')=1 & \leftarrow\mathcal{A}(\mathsf{ck})\end{array}\right]$$

Definition 4 (Code-Binding)**.** *We say that a RS code commitment* $\mathsf{CC}=(\mathsf{Setup},\mathsf{Commit},\mathsf{Prove},\mathsf{Verify})$ *is code-binding, if for every PPT algorithm* $\mathcal{A}$*, the following advantage is negligible:*

$$\Pr\left[\begin{array}{r|r}\neg(\exists c\in\mathsf{RS}^{\equiv L}:\forall i\in I,c_i=\hat{m}_i)\wedge & \mathsf{gp}\leftarrow\mathsf{Setup}(1^\lambda)\\ \forall i\in I:\mathsf{Verify}(\mathsf{gp},\mathsf{com},i,\hat{m}_i,\pi_i)=1 & (\mathsf{com},(\hat{m}_i,\pi_i)_{i\in I})\\ & \leftarrow\mathcal{A}(\mathsf{ck})\end{array}\right]$$

A.2. Polynomial Commitment Schemes

The PCS satisfies *completeness* if for any multilinear polynomial $\tilde{f}\in\mathbb{F}[X_1,\cdots,X_\mu]$ and any point $\vec{z}\in\mathbb{F}^\mu$, the following probability is 1:

$$\Pr\left[\begin{array}{r}\mathsf{pp}\leftarrow\mathsf{Setup}(1^\lambda,\mu)\\ (\mathcal{C},\mathcal{D})\leftarrow\mathsf{Commit}(\mathsf{pp},\tilde{f})\end{array}:\mathsf{Eval}(\mathsf{pp},\mathcal{C},\vec{z},\tilde{f}(\vec{z});\tilde{f},\mathcal{D})=1\right]$$

It is *binding* if for any $\mu\in\mathbb{N}$ and PPT adversary $\mathcal{A}$,

$$\Pr\left[\begin{array}{r|r}b_0=b_1=1\wedge & \mathsf{pp}\leftarrow\mathsf{Setup}(1^\lambda,\mu)\\ \tilde{f}_0\neq\tilde{f}_1 & (\mathcal{C},\tilde{f}_0,\mathcal{D}_0,\tilde{f}_1,\mathcal{D}_1)\leftarrow\mathcal{A}(\mathsf{pp})\\ & b_0\leftarrow\mathsf{OpenPoly}(\mathsf{pp},\mathcal{C},\tilde{f}_0,\mathcal{D}_0)\\ & b_1\leftarrow\mathsf{OpenPoly}(\mathsf{pp},\mathcal{C},\tilde{f}_1,\mathcal{D}_1)\end{array}\right]\leq\mathsf{negl}(\lambda)$$

The scheme satisfies *(knowledge) soundness* if Eval is an argument (of knowledge) for the relation $\mathcal{R}_{\mathsf{Eval},\mathsf{pp}}$ defined as follows:

$$\left\{(x=\{\mathcal{C},\vec{z},y\});w=\{\tilde{f},\mathcal{D}\}:\begin{array}{r}\tilde{f}(\vec{z})=y\\ \mathsf{OpenPoly}(\mathsf{pp},\mathcal{C},\tilde{f},\mathcal{D})=1\end{array}\right\}$$

Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.

Appendix B. Sumcheck Protocol

The sumcheck protocol is used to prove the sum of a multivariate polynomial $f \in \mathbb{F}^{\leq d}[X_1, \cdots, X_\mu]$ on a boolean hypercube is some public y:

$$\sum_{b_1,\cdots,b_\mu \in \{0,1\}} f(b_1, \cdots, b_\mu) = y$$

We describe the sumcheck protocol in Protocol 4. The proof size and verifier time of the protocol is $O(d\mu)$. When f is product of d multilinear polynomials [60], which is the most common case, the prover time is $d^2 \cdot 2^\mu$. The sumcheck is perfect complete and sound with $\epsilon = \frac{d\mu}{|\mathbb{F}|}$.

Protocol 4. Sumcheck Protocol.
Input. $\mathcal{P}$ holds a multivariate $f(X_1, \cdots, X_\mu)$, where each variable is of degree d. $\mathcal{V}$ is given oracle access to f, and purported sum y.

1) The i-th round, $i \in \{1, \cdots, \mu\}$:
   a) $\mathcal{P}$ sends polynomial $f_i(X) \in \mathbb{F}^{(\leq d)}[X]$ to $\mathcal{V}$, where f_i is defined as follows:
   $$f_i(X) := \sum_{\vec{b}_{[i+1:]} \in \{0,1\}^{\mu-i}} f\left(\vec{r}_{[:i-1]}, X, \vec{b}_{[i+1:]}\right)$$
   b) $\mathcal{V}$ checks $f_i(0) + f_i(1) = y$ when $i = 1$; otherwise $\mathcal{V}$ checks $f_i(0) + f_i(1) = f_{i-1}(r_{i-1})$.
   c) $\mathcal{V}$ samples $r_i \in \mathbb{F}$ and sends r_i to $\mathcal{P}$.
2) $\mathcal{V}$ queries oracle of f to obtain $f(\vec{r})$ and checks $f_\mu(r_\mu) = f(\vec{r})$. $\mathcal{V}$ outputs 1 if all the checks pass, otherwise outputs 0.

Appendix C. Evaluation Reduction

We define the definition of *evaluation reduction* based on *reduction of knowledge* in [61]. First, we define an evaluation relation $\mathcal{R}_{\mathsf{Eval}}$ as follows:

$$\left\{\left(\mathcal{C}_f, \vec{z}, y; \tilde{f}\right) : \tilde{f}(\vec{z}) = y \wedge \mathcal{C}_f = \mathsf{PC.Commit}(\tilde{f})\right\}$$

where $\tilde{f}$ is a μ-variate multilinear polynomial, $\vec{z} \in \mathbb{F}^\mu, y \in \mathbb{F}$. Evaluation reduction is a public-coin interactive protocol between $\mathcal{P}$ and $\mathcal{V}$, denoted as

$$(\vec{r}, y_r) \leftarrow \langle \mathcal{P}(\tilde{f}), \mathcal{V} \rangle (\mathcal{C}_f, \vec{z}, y)$$

where $\vec{r} \in \mathbb{F}^\mu, y_r \in \mathbb{F}$. The evaluation reduction protocol satisfies the following properties:

- Completeness: for any $\left(\mathcal{C}_f, \vec{z}, y; \tilde{f}\right) \in \mathcal{R}_{\mathsf{Eval}}$, the following probability is 1:
$$\Pr\left[(\vec{r}, y_r) \leftarrow \langle \mathcal{P}(\tilde{f}), \mathcal{V} \rangle (\mathcal{C}_f, \vec{z}, y) : \left(\mathcal{C}_f, \vec{r}, y_r; \tilde{f}\right) \in \mathcal{R}_{\mathsf{Eval}}\right]$$

Protocol 5. Multilinear Evaluation Reduction.
Public input: a point $\vec{z} = (z_1, \ldots, z_\mu)$ and a purported evaluation $y = \tilde{f}(\vec{z})$.
Prover witness: the polynomial $\tilde{f}$.
Verifier output: a random point $\vec{r} \in \mathbb{F}^\mu$ and value $y_r = \tilde{f}(\vec{r})$.

1) $\mathcal{P}$ sends $l_1(X) = \tilde{f}(X, \vec{z}_{[2:]})$ to $\mathcal{V}$.
2) $\mathcal{V}$ checks $l_1(z_1) = y$, aborts if false.
3) For each $i = 2$ to μ:
   a) $\mathcal{V}$ samples $r_{i-1} \in \mathbb{F}$ and sends it to $\mathcal{P}$.
   b) $\mathcal{P}$ responds with $l_i(X) = \tilde{f}(\vec{r}_{[:i-1]}, X, \vec{z}_{[i+1:]})$.
   c) $\mathcal{V}$ checks $l_{i-1}(r_{i-1}) = l_i(z_i)$, aborts if false.
4) $\mathcal{V}$ samples r_μ and outputs $(\vec{r}, y_r)$ where $y_r = l_\mu(r_\mu)$.

- Soundness: for any PPT prover $\mathcal{P}^*$, the following probability is *negligible*:
$$\Pr\left[\begin{array}{l}(\vec{r}, y_r) \leftarrow \langle \mathcal{P}(\tilde{f}), \mathcal{V} \rangle (\mathcal{C}_f, \vec{z}, y) \\ \quad \wedge \left(\mathcal{C}_f, \vec{r}, y_r; \tilde{f}\right) \in \mathcal{R}_{\mathsf{Eval}}\end{array} : \left(\mathcal{C}_f, \vec{z}, y; \tilde{f}\right) \notin \mathcal{R}_{\mathsf{Eval}}\right]$$

Informally, the soundness states that if the original claim of $\tilde{f}(\vec{z}) = y$ isn't correct, with overwhelming probability, the reduced evaluation $f(\vec{r}) \neq y_r$.

The formal description of our evaluation reduction is presented in Theorem 5.

Appendix D. Proof of Theorems

D.1. Theorem 2

Proof. Define $f_i = p_i(r_i)$, where $p_i(X)$ is the univariate polynomial sent to $\mathcal{V}$ in the i-th round. For simplicity, we assume s is divisible by μ. Define $d = \frac{\mu}{s}$ and $f_0 = y$. For any $i \in [1, d]$,

$$\begin{aligned}
&\Pr\left[\tilde{f}(\vec{r}_{[:i\cdot s]}, \vec{z}_{[i\cdot s+1:]}) = f_i\right] \\
\leq &\Pr\left[\begin{array}{l}\tilde{f}(\vec{r}_{[:i\cdot \ell]}, \vec{z}_{[i\cdot s+1:]}) \\ \quad = p_i(r_i)\end{array} \middle| \begin{array}{l}\tilde{f}(\vec{r}_{[:(i-1)s]}, X^{2^0}, \cdots, X^{2^{s-1}}, \vec{z}_{[i\cdot s+1:]}) \\ \quad \neq p_i(X)\end{array}\right] \\
&+ \Pr[\tilde{f}(\vec{r}_{[:(i-1)\cdot s]}, X^{2^0}, \cdots, X^{2^{s-1}}, \vec{z}_{[i\cdot s+1:]}) = p_i(X)] \\
\leq &\frac{2^s}{|\mathbb{F}|} + \Pr[\tilde{f}(\vec{r}_{[:(i-1)\cdot s]}, \vec{z}_{[(i-1)\cdot s+1:]}) = f_{i-1}]
\end{aligned}$$

Thus we have

$$\begin{aligned}
&\Pr[\tilde{f}(\vec{r}_{[:i\cdot s]}, \vec{z}_{[i\cdot s+1:]}) = f_i] \\
&\qquad - \Pr[\tilde{f}(\vec{r}_{[:(i-1)\cdot s]}, \vec{z}_{[(i-1)s+1:]}) = f_{i-1}] \leq \frac{2^s}{|\mathbb{F}|} \quad (1)
\end{aligned}$$

For $i \in [\mu]$, we sum up (1) and notice that $\Pr[\tilde{f}(\vec{z}) = f_0] = 0$.

$$\Pr[y_r = \tilde{f}(\vec{r})] = \Pr[y_r = \tilde{f}(\vec{r})] - \Pr[\tilde{f}(\vec{z}) = f_0] \leq \frac{\mu \cdot 2^s}{s \cdot |\mathbb{F}|}$$

Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.

□

D.2. Theorem 3

Proof. We first assume PST, sumcheck and Merkle tree has perfect soundness.

We call a vector $\vec{v}$ *approximates* RS code RS, if and only if there exists codeword $\vec{c} \in \mathsf{RS}$ such that $\Delta(\vec{v}, \vec{c}) < 1 - \rho$, where $\Delta(\cdot, \cdot)$ represents relative hamming distance of two vectors. Let $\vec{v}'$ be the vector folded from the committed matrix C. Let $\vec{f'}$ be the codeword corresponding to the PST committed multilinear polynomial. If $\vec{v}'$ doesn't approximate $\vec{f'}$, which means $\Delta(\vec{v}', \vec{f'}) \geq 1 - \rho$, then for each random sample $x \in [2^{\mu - \mu'}]$,

$$\Pr\left[\vec{v}'[x] = \vec{f'}[x]\right] \leq \rho$$

Then for s independent random samples,

$$\Pr\left[\forall x \in I, \vec{v}'[x] = \vec{f'}[x]\right] \leq \rho^s$$

Consider the negligible soundness error from Merkle tree and PST batched open, the overall soundness error is $\rho^s + \mathsf{negl}(\lambda)$. □

Appendix E. Meta-Review

The following meta-review was prepared by the program committee for the 2026 IEEE Symposium on Security and Privacy (S&P) as part of the review process as detailed in the call for papers.

E.1. Summary

This work presents efficient (batched) Reed-Solomon (RS) code commitments, which are commitments to (batches of) RS codes such that later any set of indices of the code can be proven with respect to the commitment. Such commitments are building blocks for VSS schemes and data-availability schemes.

It proposes a new RS code commitment scheme, UltraProofs. UltraProofs works by batching multiple polynomial evaluation checks into a single evaluation check, and lifts this efficiently to proving multiple evaluations of multiple RS codes.

This work additionally designs a new polynomial commitment scheme, LightLigero, for proving the final batched polynomial evaluation claim efficiently. Finally, the paper benchmarks both LightLigero individually as well as UltraProofs in the context of VSS and data availability.

E.2. Scientific Contributions

- Creates a New Tool to Enable Future Science.
- Provides a Valuable Step Forward in an Established Field.

E.3. Reasons for Acceptance

1) The paper proposes a new batching scheme for multivariate polynomials, a new polynomial commitment scheme, and a way to lift this to RS code commitments. Additionally, the paper shows how to apply these to two applications of general interest (VSS and DAS).
2) The core result of compressing polynomial evaluation claims is new and general enough to be potentially useful in other contexts as well.

Authorized licensed use limited to: New Mexico State University. Downloaded on October 12,2026 at 21:12:49 UTC from IEEE Xplore. Restrictions apply.